# *Entrega #1 - Estudio Tienda Virtual GRUPO #2*

*David Santiago García Preciado*

*Fernando Javier Alvarado Arispe*

*Jorge Orlando Jaramillo Nieves*

*Universidad El Bosque*

*Facultad de ingeniería*

*Ingeniería de Sistemas*

*Sistemas Distribuidos*

*14/03/2026*

## TABLA DE CONTENIDO

**1. División en microservicios, despliegue y pruebas en Postman..... 4**
1.1 Descripción general del proyecto.....4
1.2 Infraestructura con máquinas virtuales Ubuntu.....5
1.3 Orden de arranque del sistema.....10
1.4 Pruebas de funcionamiento en Postman..... 12
1.4.1 Authentication Service – Login y gestión de usuarios.....12
1.4.2 Client Service – Gestión de clientes.....15
1.4.3 Supplier Service – Gestión de proveedores.....19
1.4.4 Catalog Service – Gestión de productos.....22
1.4.5 Buy Service – Registro de compras a proveedores.....26
1.4.6 Sale Service – Registro de ventas..... 28
1.4.7 Prueba de seguridad – Acceso sin token.....31
**2. Funcionalidad de Microservicios e Inconvenientes Superados.....33**
2.1 ¿Qué es un Microservicio?.....33
2.2 Microservicio Eureka Server..... 33
Funcionalidad..... 33
Inconvenientes superados..... 34
2.3 Microservicio API Gateway..... 34
Funcionalidad..... 34
Inconvenientes superados..... 35
2.4 Microservicio: Authentication Service.....35
Funcionalidad..... 36
Inconvenientes superados..... 37
2.5 Microservicio: Client Service..... 37
Funcionalidad..... 37
Inconvenientes superados..... 38
2.6 Microservicio: Supplier Service..... 38
Funcionalidad..... 39
Inconvenientes superados..... 39
2.7 Microservicio: Catalog Service.....40
Funcionalidad.....40
Inconvenientes superados..... 41
2.8 Microservicio: Buy Service..... 41
Funcionalidad..... 41
Inconvenientes superados..... 42
2.9 Microservicio: Sale Service..... 42
Funcionalidad..... 42
Inconvenientes superados..... 43
**3. Arquitectura Utilizada e Implementación.....44**
3.1 ¿Qué arquitectura se utilizó?..... 44
3.2 Componentes de la arquitectura y cómo se implementaron.....44

3.3 Comunicación entre microservicios en distintas VMs..... 45
3.4 Seguridad de la arquitectura..... 46
3.5 Frontend con React Vite.....46
3.6 Justificación de cambios tecnológicos respecto al documento original..... 47
3.7 Patrones de diseño aplicados..... 48

# 1. División en microservicios, despliegue y pruebas en Postman

## 1.1 Descripción general del proyecto

El proyecto consiste en el software de una tienda genérica virtual desarrollada con la arquitectura de microservicios. En lugar de seguir la especificación original que planteaba una arquitectura monolítica con JSF y JSP, se tomó la decisión de modernizar el stack tecnológico utilizando Spring Boot y una arquitectura de microservicios.

El sistema está compuesto por 8 microservicios independientes, cada uno desplegado en su propia máquina virtual Ubuntu 22.04.5 LTS con Java, Maven, Tomcat y MySQL instalados. Los microservicios tienen además su propia base de datos MySQL corriendo localmente dentro de la misma VM.

| VM | Microservicio | Puerto app | MySQL (local) | Base de datos |
|---|---|---|---|---|
| VM 1 | eureka-server | 8761 | N/A | N/A |
| VM 2 | api-gateway | 8080 | N/A | N/A |
| VM 3 | authentication-service | 8081 | 3306 | tienda_aut_db |
| VM 4 | client-service | 8082 | 3306 | tienda_cli_db |
| VM 5 | supplier-service | 8083 | 3306 | tienda_sup_db |
| VM 6 | catalog-service | 8084 | 3306 | tienda_cat_db |
| VM 7 | buy-service | 8085 | 3306 | tienda_buy_db |
| VM 8 | sale-service | 8086 | 3306 | tienda_sal_db |

El frontend de la aplicación es una SPA (Single Page Application) desarrollada con React y Vite, que se comunica con el backend exclusivamente a través del API Gateway en el puerto 8080 mediante peticiones HTTP REST.

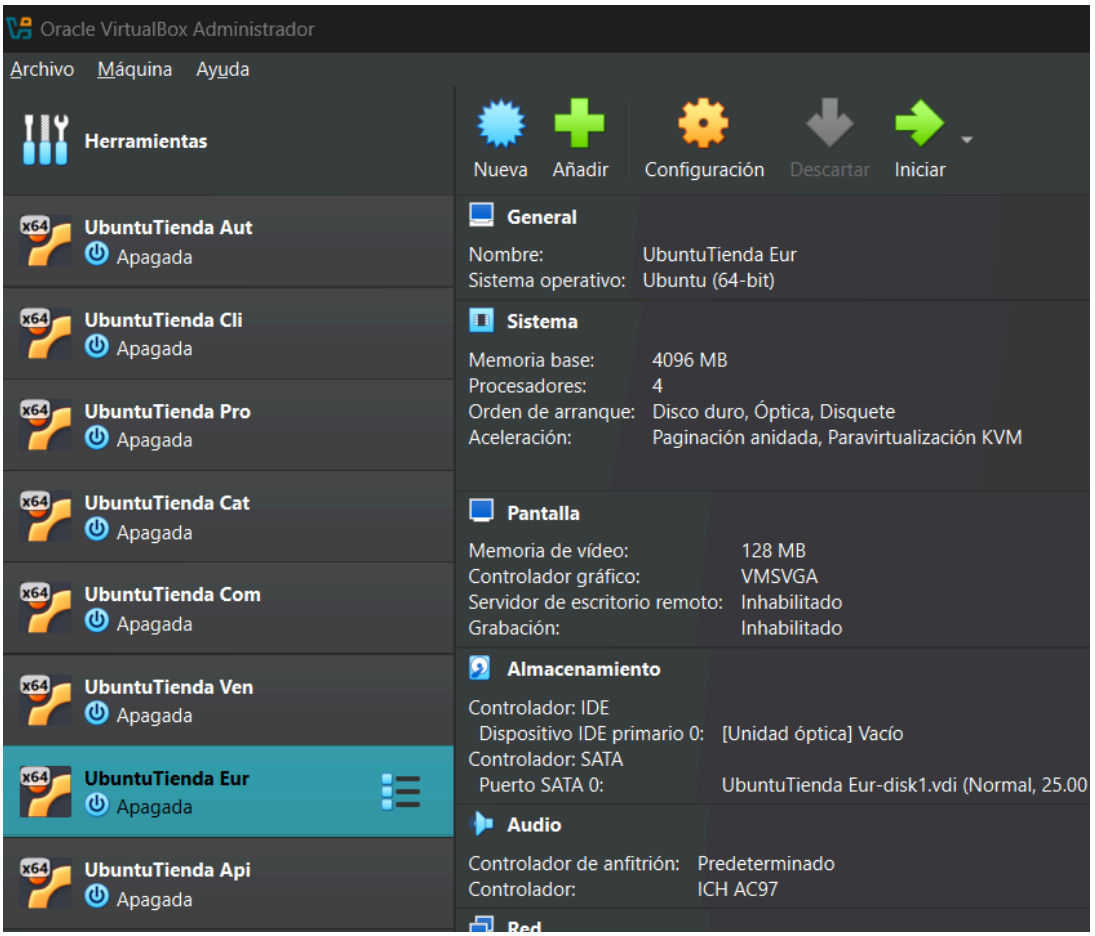
## 1.2 Infraestructura con máquinas virtuales Ubuntu

Cada microservicio vive en su propio entorno aislado. Dentro de cada VM se realizó la siguiente instalación base:

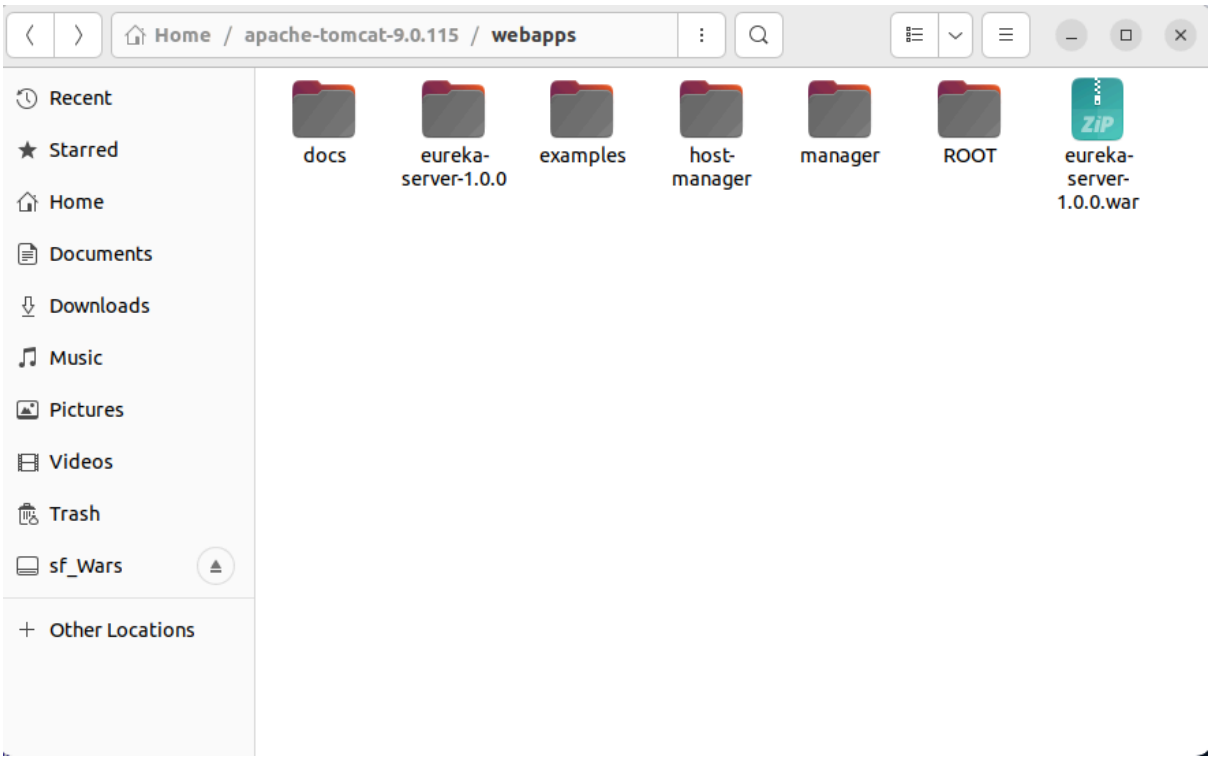
- Ubuntu 22.04.5 LTS
- JDK 17 para ejecutar las aplicaciones Spring Boot
- Apache Tomcat 10 para servir los archivos WAR de cada microservicio
- MySQL 8 (únicamente en las VMs de los servicios de negocio, VM 3 a VM 8)
- Maven 9.3.11 (únicamente en la etapa de desarrollo antes de generar los war)

Una vez preparada la VM, el proceso de despliegue de cada microservicio consiste en compilar el proyecto con Maven para generar el archivo WAR, copiarlo al directorio webapps de Tomcat dentro de la VM correspondiente, y arrancar Tomcat. La base de datos se inicia ejecutando los scripts SQL de creación e inserción de datos semilla antes del primer arranque del servicio.

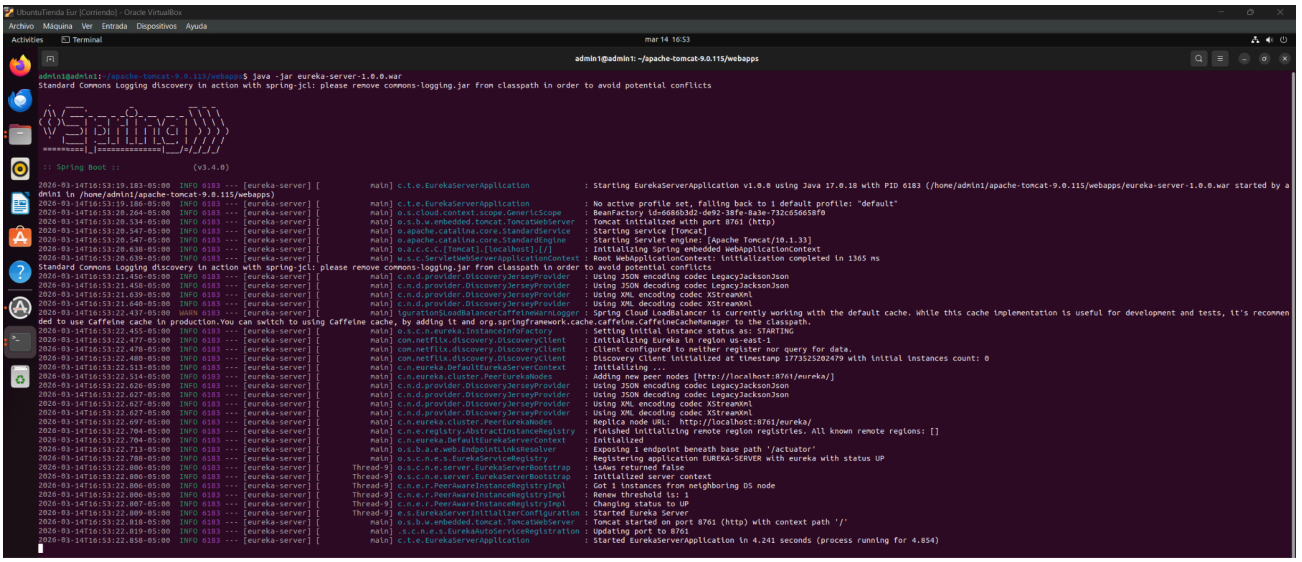
Máquinas virtuales:



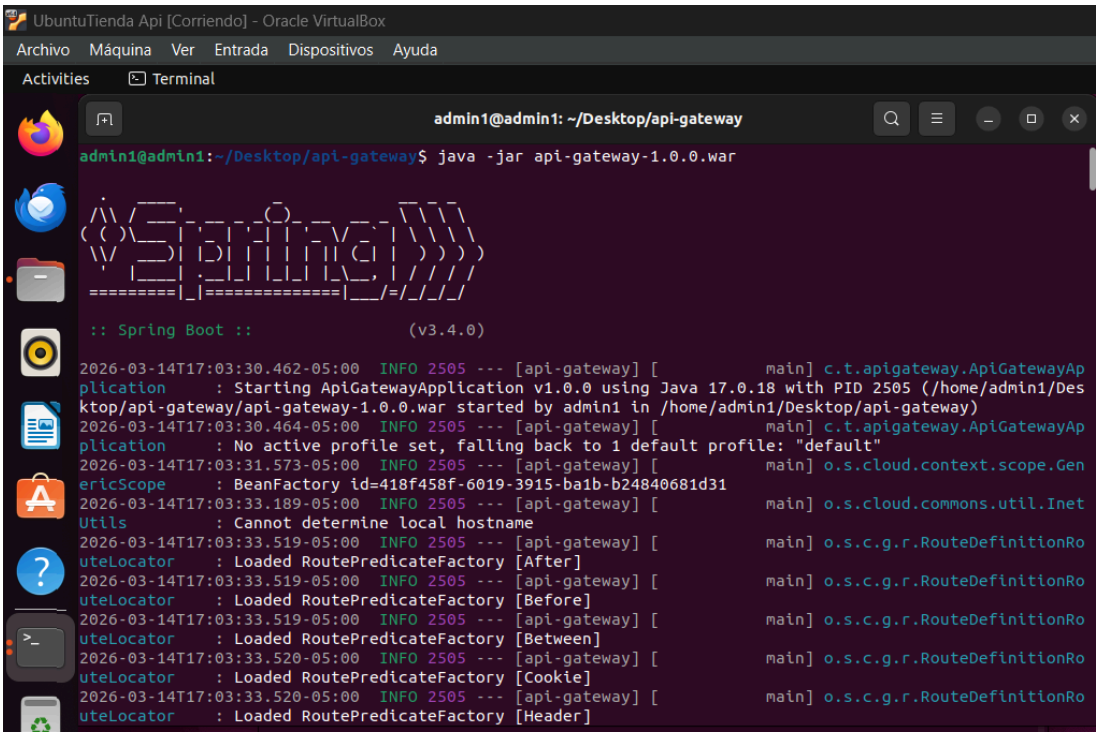
Archivo .war en la máquina virtual



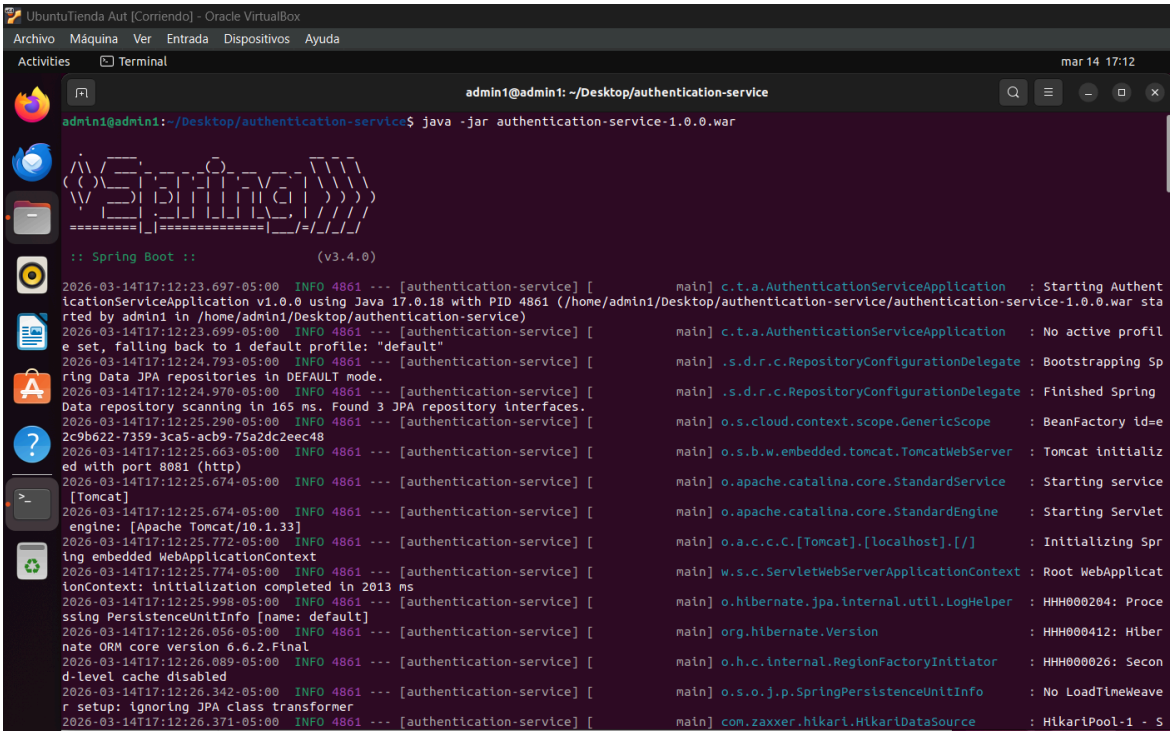

A continuación se puede ver la ejecución del archivo .war de eureka-server



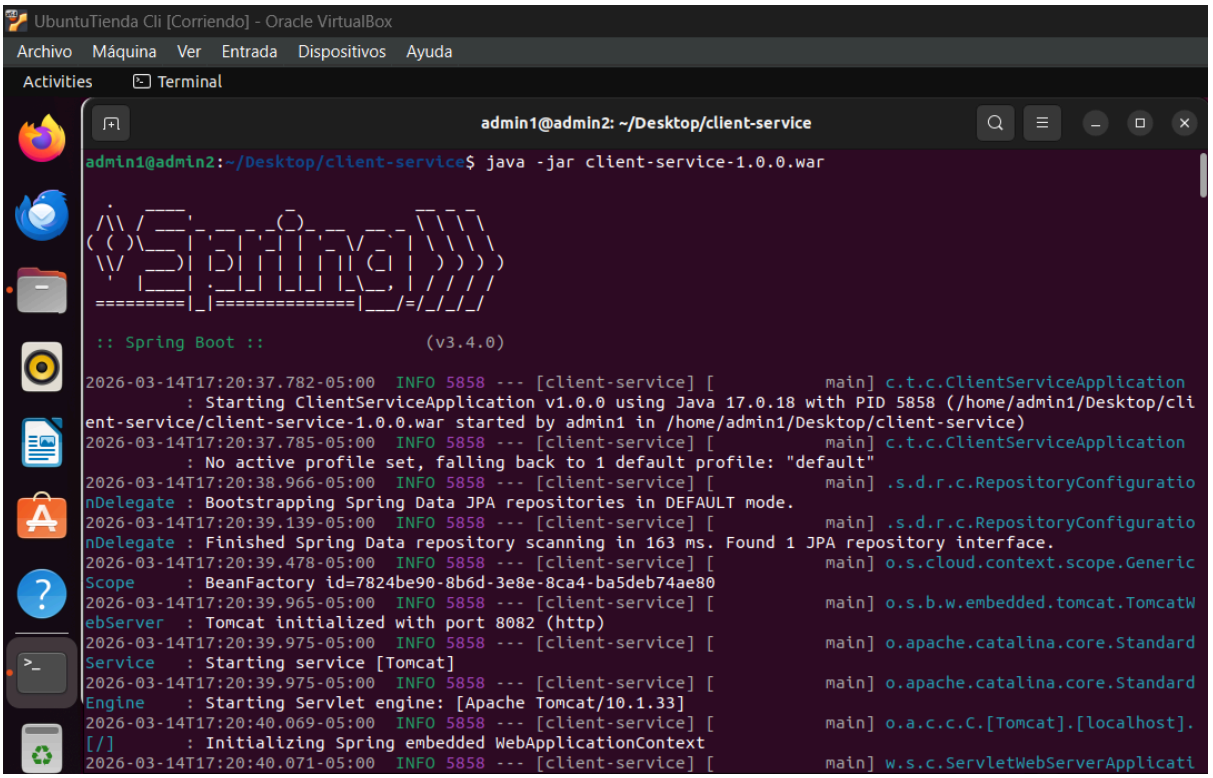
Máquina virtual de UbuntuTienda Api



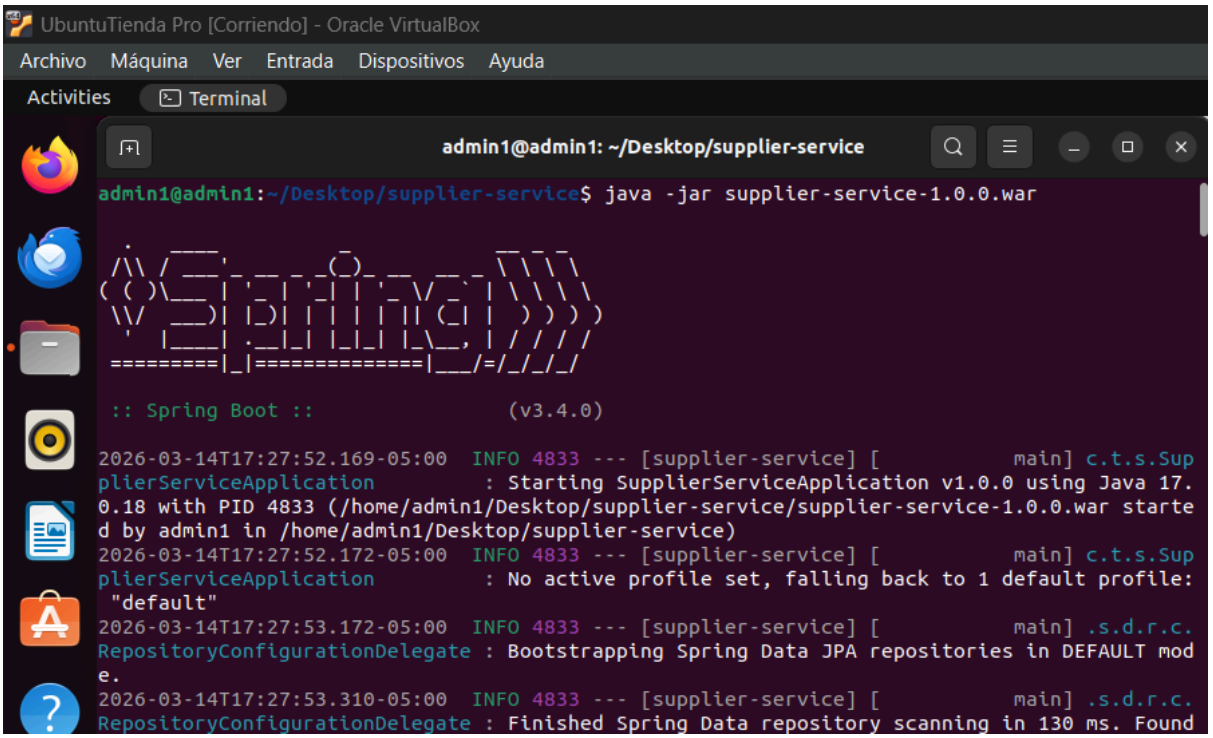

Máquina virtual de UbuntuTienda Aut



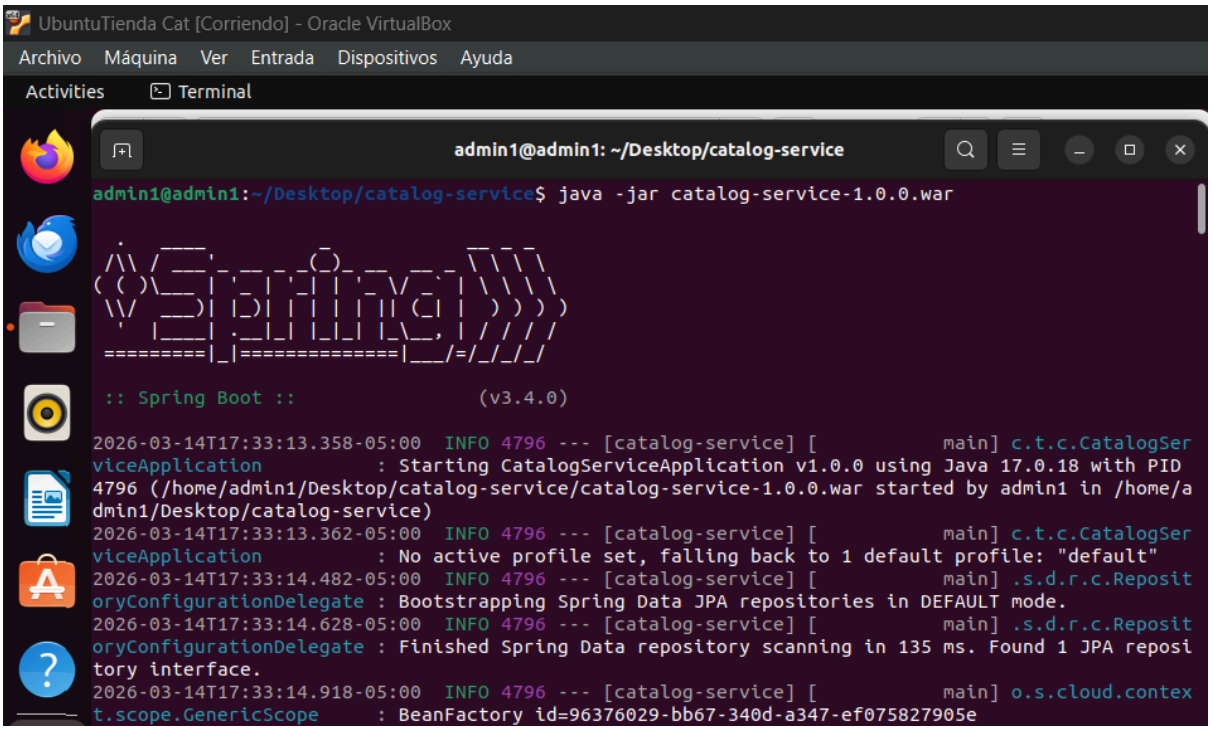
Máquina virtual de UbuntuTienda Clie



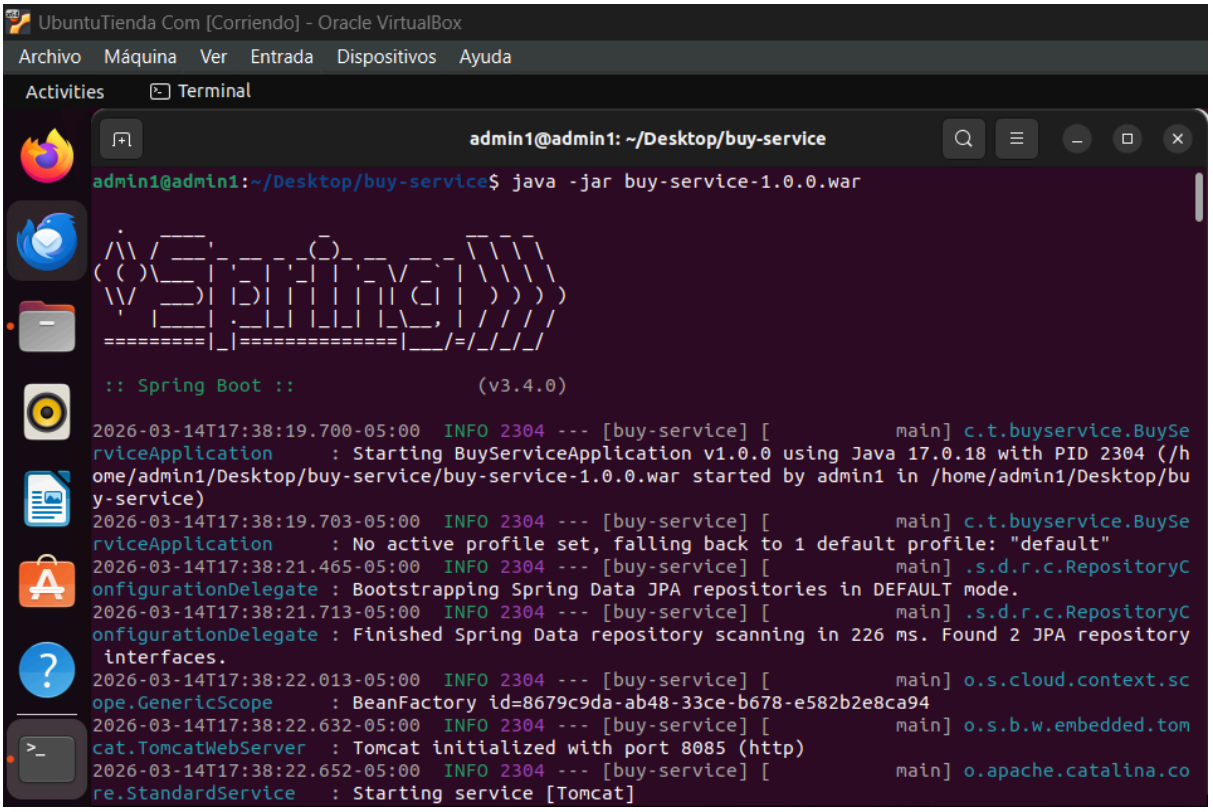

Máquina virtual de UbuntuTienda Pro



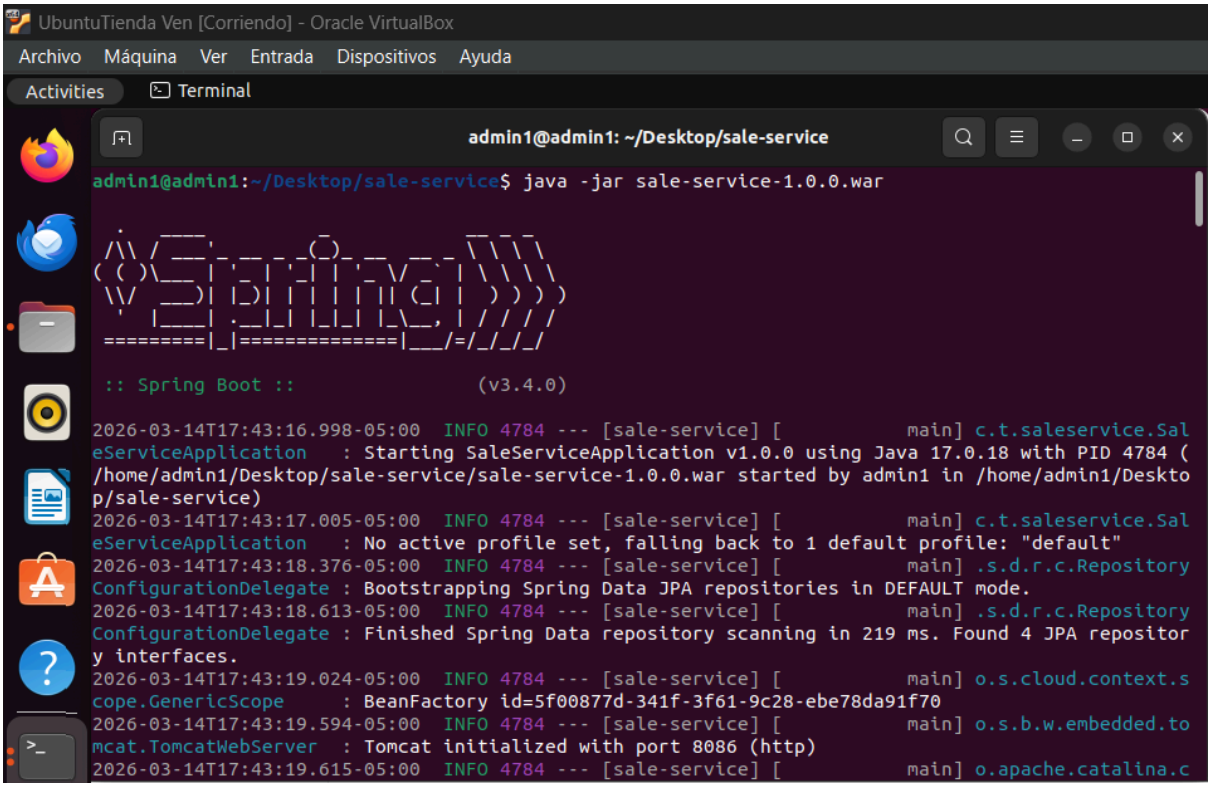
Máquina virtual de UbuntuTienda Cat




Máquina virtual de UbuntuTienda Com




Máquina virtual de UbuntuTienda Ven



Dado que cada microservicio se conecta a su propia base de datos local, el application.properties de cada servicio apunta a localhost:3306, ya que MySQL corre en la misma máquina (por ejemplo: spring.datasource.url=jdbc:mysql://localhost:3306/tienda_cli_db para el microservicio de clientes).

Esto simplifica la configuración de red entre servicios, ya que la única comunicación entre VMs distintas ocurre cuando un microservicio llama a otro a través del API Gateway, usando las IPs de las VMs en la red local.

## 1.3 Orden de arranque del sistema

Dado que los microservicios dependen entre sí para registrarse y descubrirse, el orden de arranque es importante:

1. VM 1: Arrancar Tomcat con el eureka-server, y este debe estar listo antes que cualquier otro servicio, ya que eureka se encarga de registrar los servicios
2. VM 2: Arrancar Tomcat con el api-gateway, este se registra en Eureka y queda listo para enrutar los servicios
3. VM3 - VM8: El resto de VMs en cualquier orden (authentication-service, client-service, supplier-service, catalog-service, buy-service, sale-service)

Instances currently registered with Eureka

| Application | AMIs | Availability Zones | Status |
|---|---|---|---|
| API-GATEWAY | n/a (1) | (1) | UP (1) - 192.168.56.101:api-gateway:8080 |
| AUTHENTICATION-SERVICE | n/a (1) | (1) | UP (1) - 192.168.56.102:authentication-service:8081 |
| BUY-SERVICE | n/a (1) | (1) | UP (1) - 192.168.56.103:buy-service:8085 |
| CATALOG-SERVICE | n/a (1) | (1) | UP (1) - 192.168.56.104:catalog-service:8084 |
| CLIENT-SERVICE | n/a (1) | (1) | UP (1) - 192.168.56.105:client-service:8082 |
| SALE-SERVICE | n/a (1) | (1) | UP (1) - 192.168.56.106:sale-service:8086 |
| SUPPLIER-SERVICE | n/a (1) | (1) | UP (1) - 192.168.56.107:supplier-service:8083 |

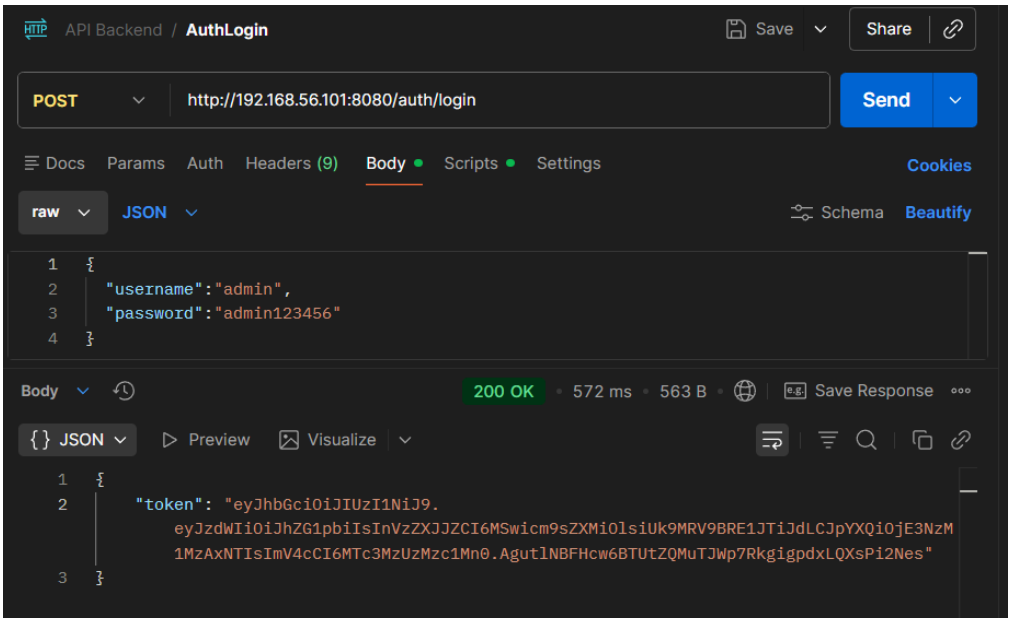
## 1.4 Pruebas de funcionamiento en Postman

Aquí se encuentra documentado el funcionamiento de cada microservicio verificado mediante Postman. Todos los endpoints (excepto login y signup) requieren un token JWT en el header Authorization: Bearer <token>. El token se obtiene primero haciendo login contra el API Gateway, que enruta la petición al authentication-service.

### 1.4.1 Authentication Service – Login y gestión de usuarios

**Obtener token JWT (login):**

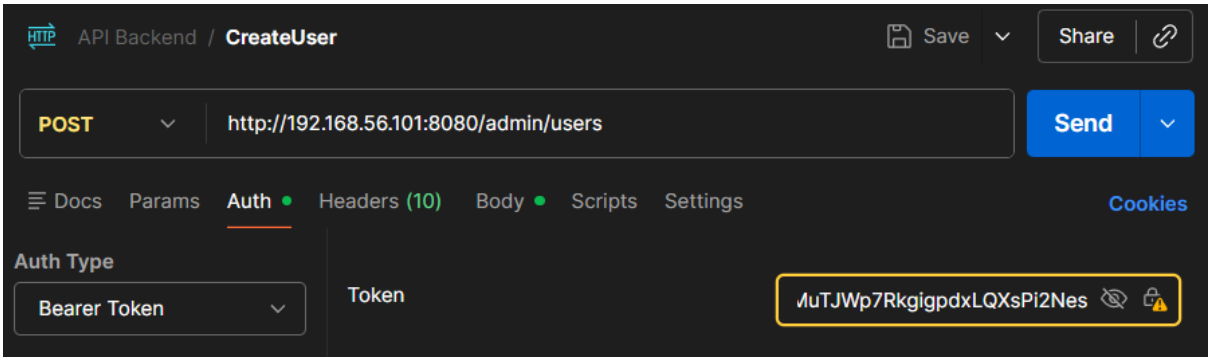
Se envía una petición POST a http://192.168.56.101:8080/auth/login con el body:

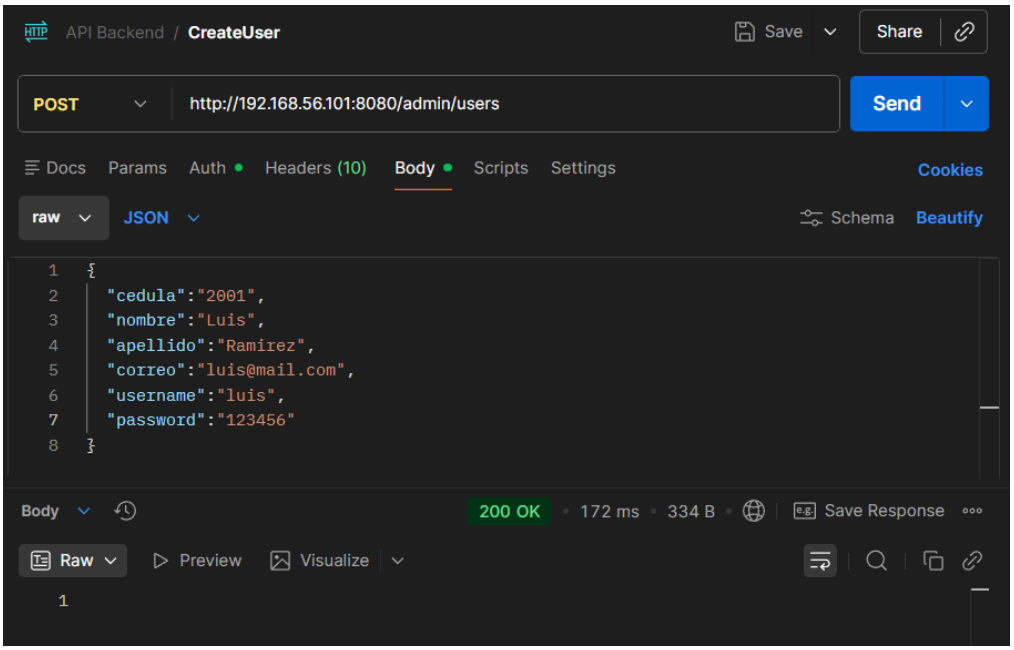



El sistema responde con el token JWT que se usará en todas las demás peticiones.

**Crear un nuevo usuario:**

Se envía POST a http://192.168.56.101:8080/admin/users con el header Authorization: Bearer <token> y los datos del nuevo usuario en el body.




**Listar todos los usuarios:**

GET a http://192.168.56.101:8080/admin/users con el token en el header.

GET http://192.168.56.101:8080/admin/users Send

Docs Params Auth Headers (8) Body Scripts Settings Cookies

Auth Type

Be...

Token

The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.

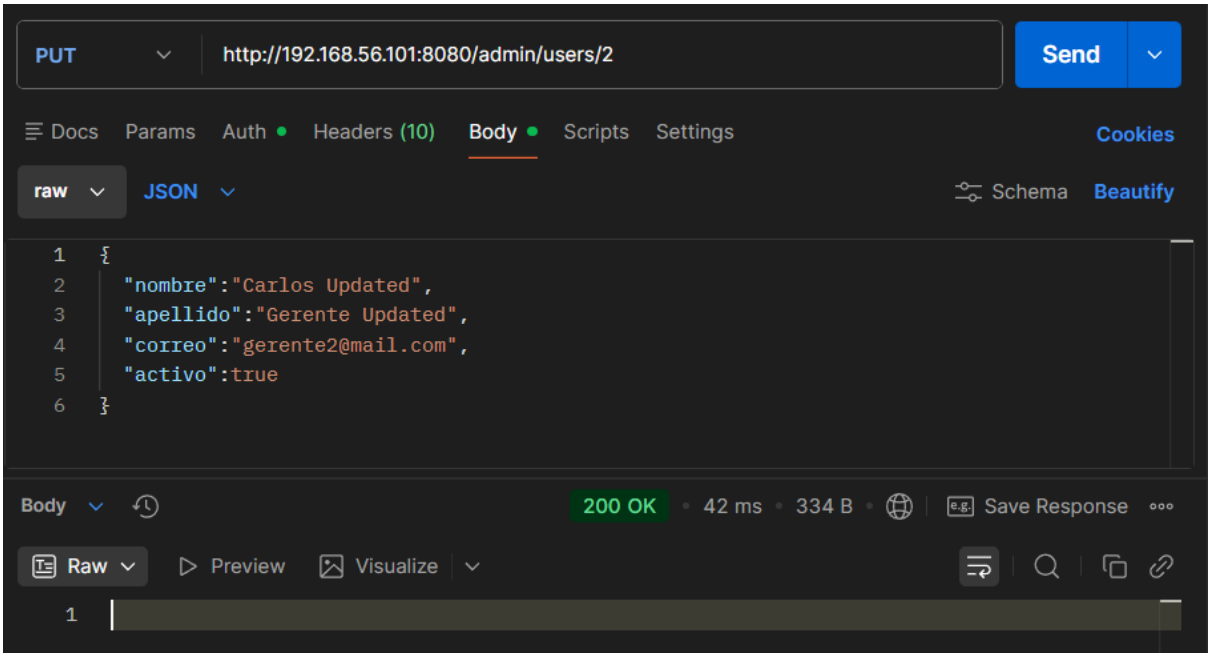
Body 200 OK 38 ms 1.25 KB Save Response

JSON Preview Visualize


```
[
    {
        "id": 1,
        "cedula": "1001",
        "nombre": "Admin",
        "apellido": "Principal",
        "correo": "admin@tienda.com",
        "username": "admin",
        "activo": true,
        "roles": [
            "ROLE_ADMIN"
        ]
    },
    {
        "id": 2,
        "cedula": "1002",
        "nombre": "Carlos",
        "apellido": "Delgado",
        "correo": "gerente@tienda.com",
        "username": "gerente",
        "activo": true,
        "roles": [
            "ROLE_GERENTE"
        ]
    },
```

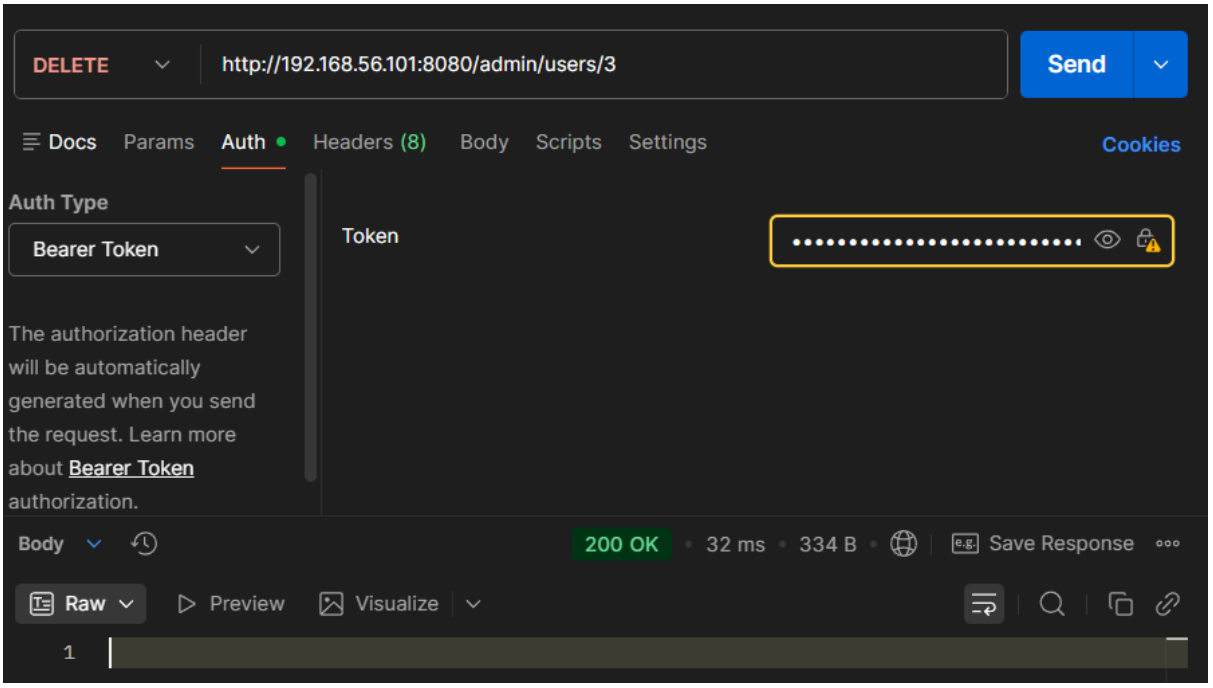

**Actualizar un usuario:**

PUT a http://192.168.56.101:8080/admin/users/{id} con los datos a modificar.




**Desactivar un usuario (borrado lógico):**

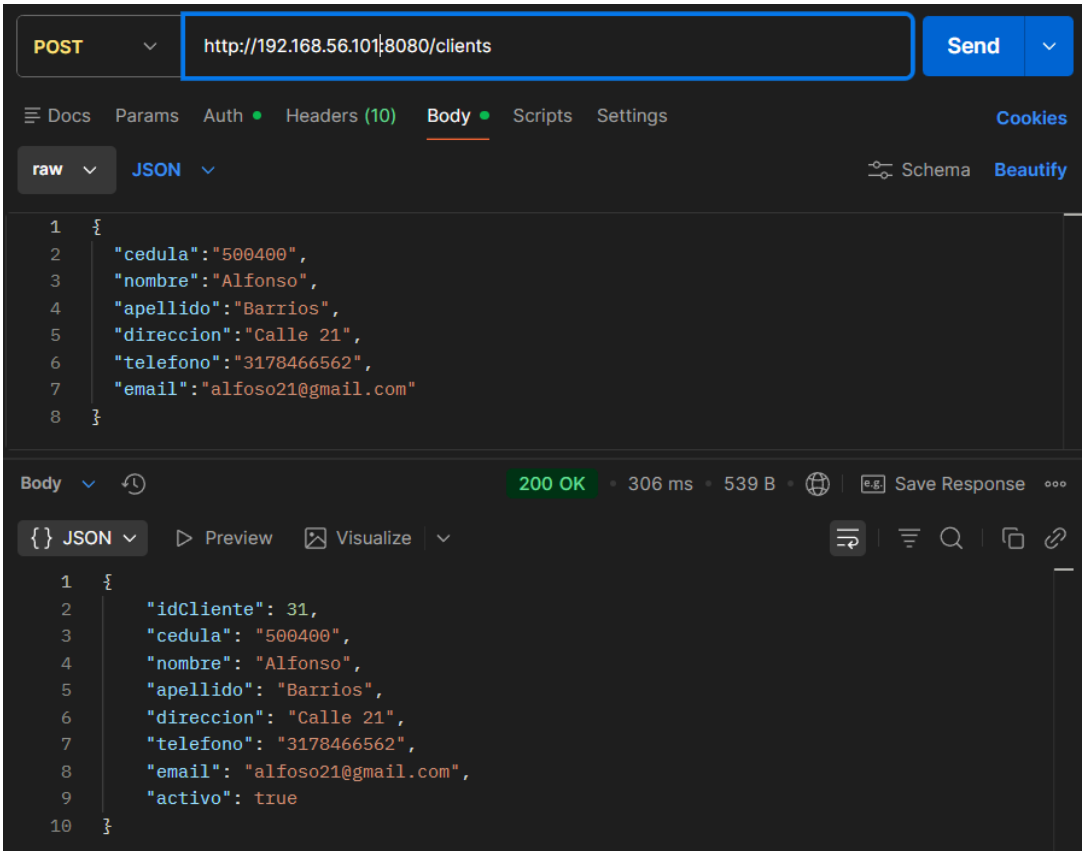
DELETE a http://192.168.56.101:8080/admin/users/{id}/act.




## 1.4.2 Client Service – Gestión de clientes

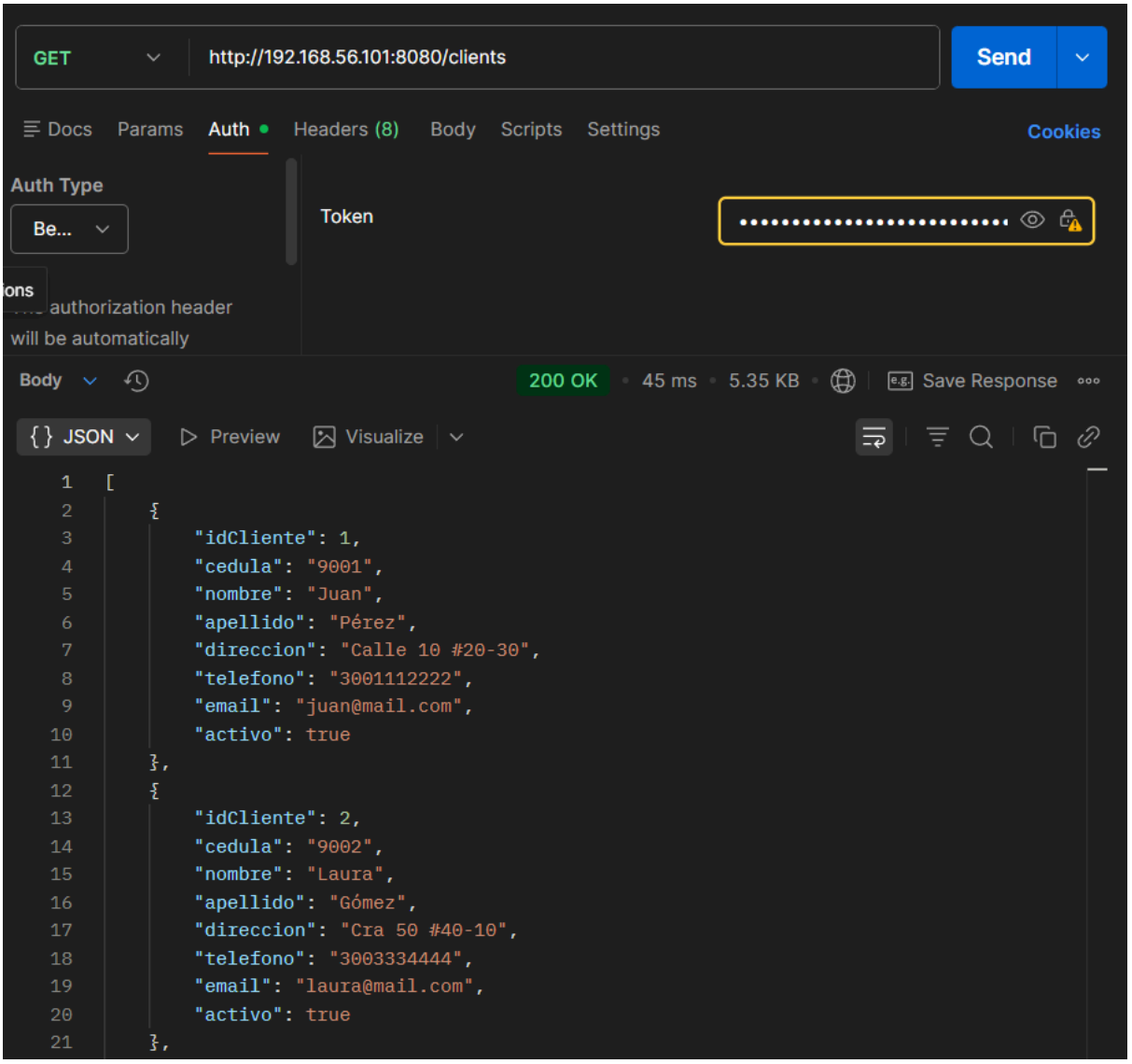
**Crear un cliente:**

POST a http://192.168.56.101:8080/clients con los datos del cliente en el body.



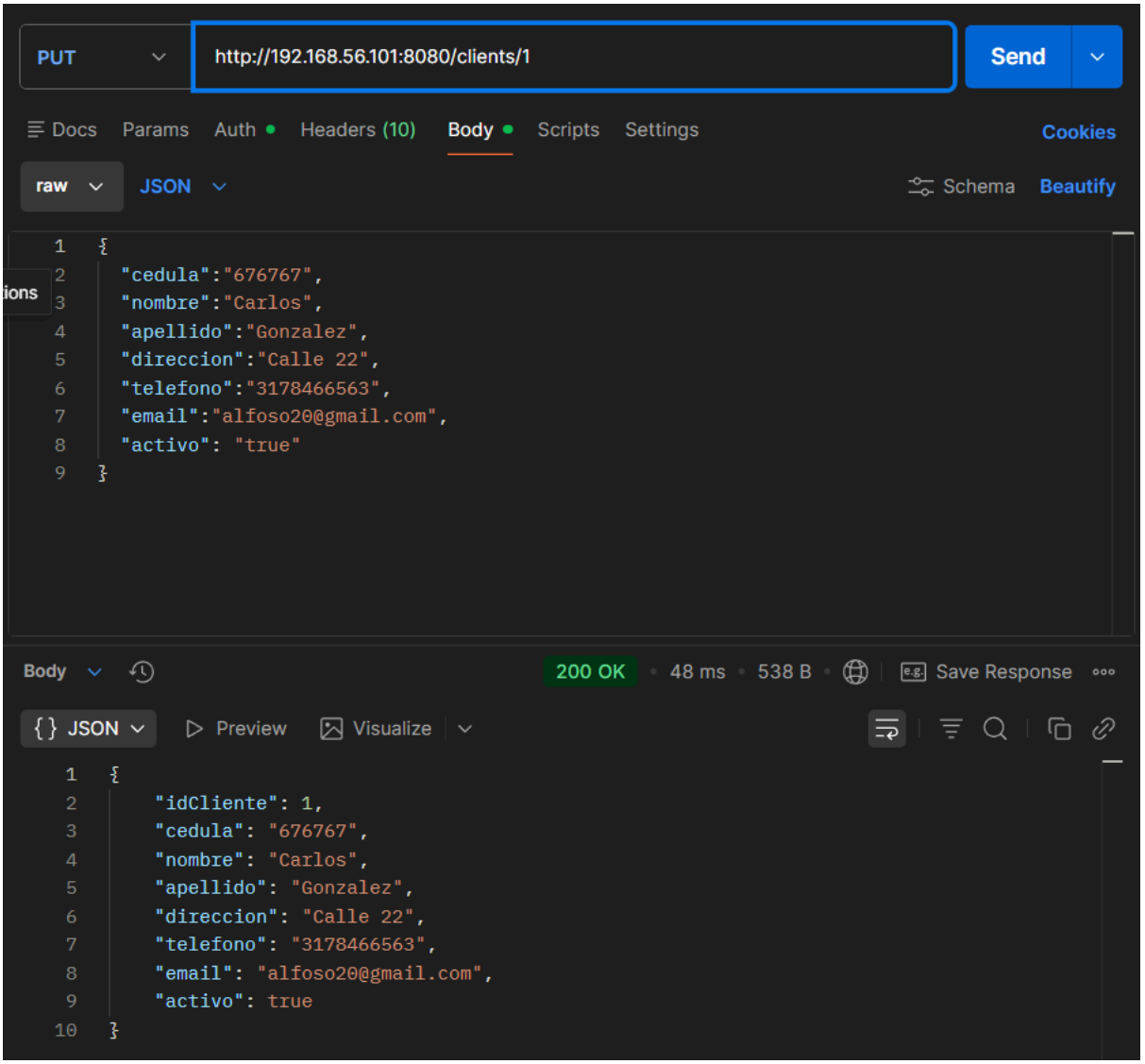

**Listar todos los clientes:**

GET a http://192.168.56.101:8080/clients.



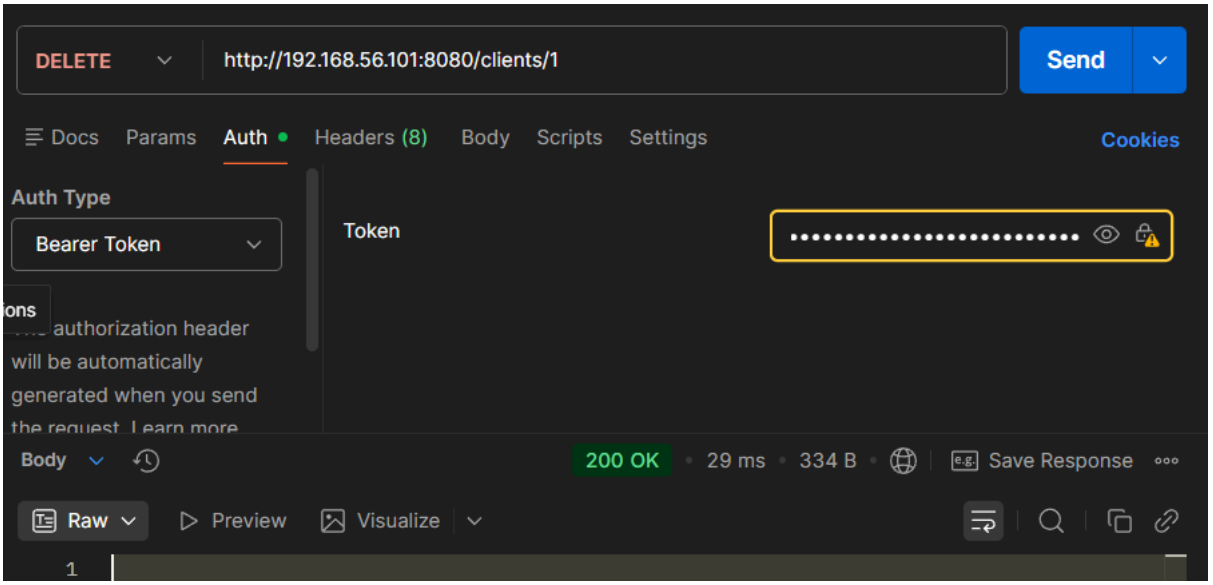

**Actualizar un cliente:**

PUT a http://192.168.56.101:8080/clients/{id} con los datos modificados.




**Eliminar un cliente:**

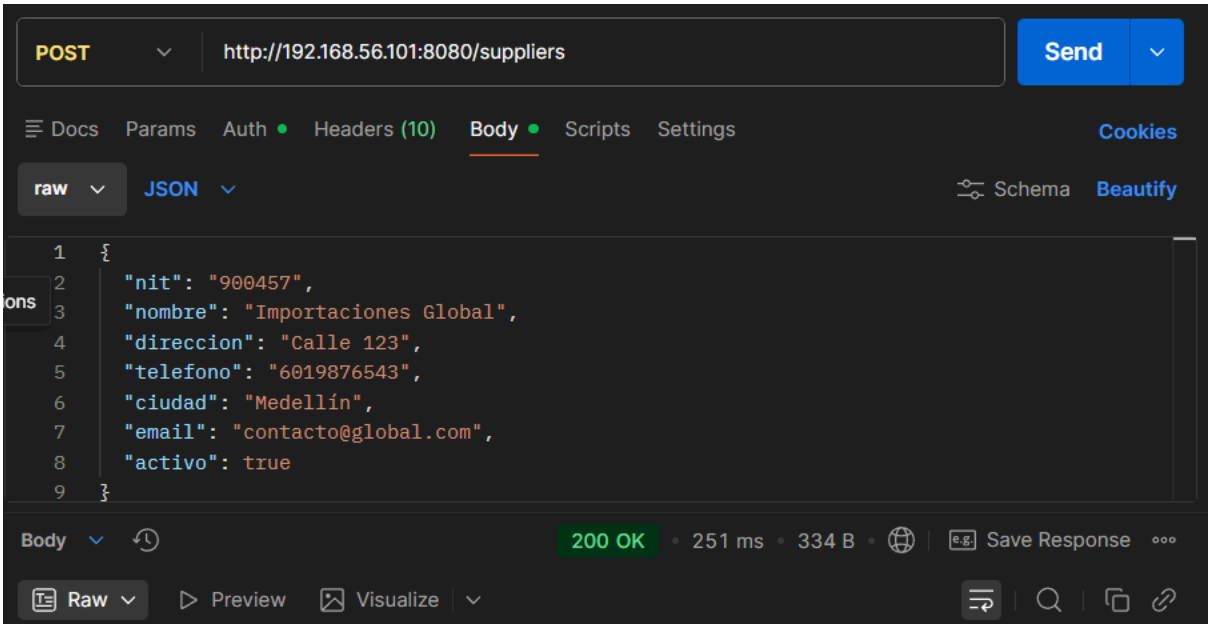
DELETE a http://192.168.56.101:8080/clients/{id}.




### 1.4.3 Supplier Service – Gestión de proveedores

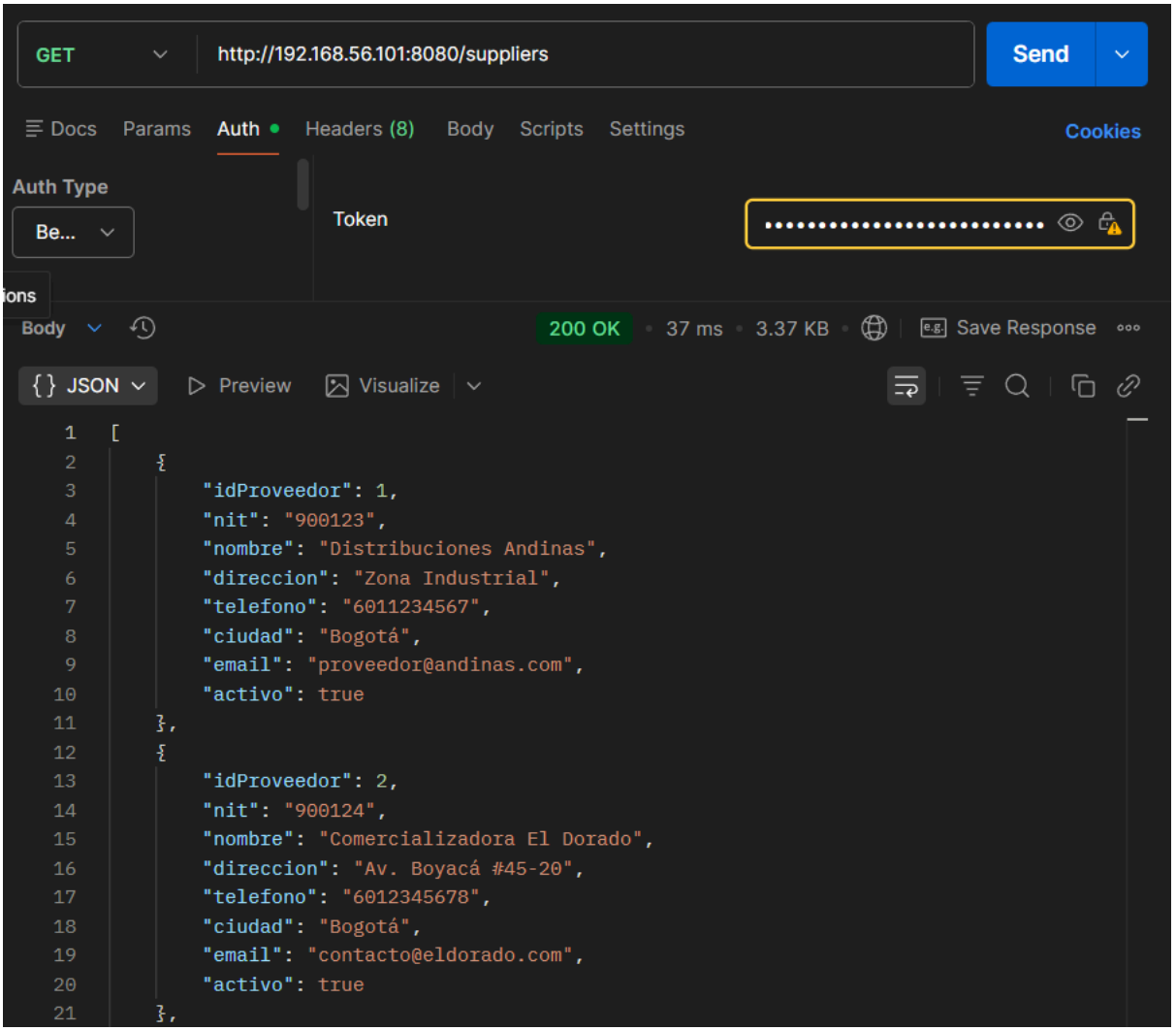
**Crear un proveedor:**

POST a http://192.168.56.101:8080/suppliers con los datos del proveedor.



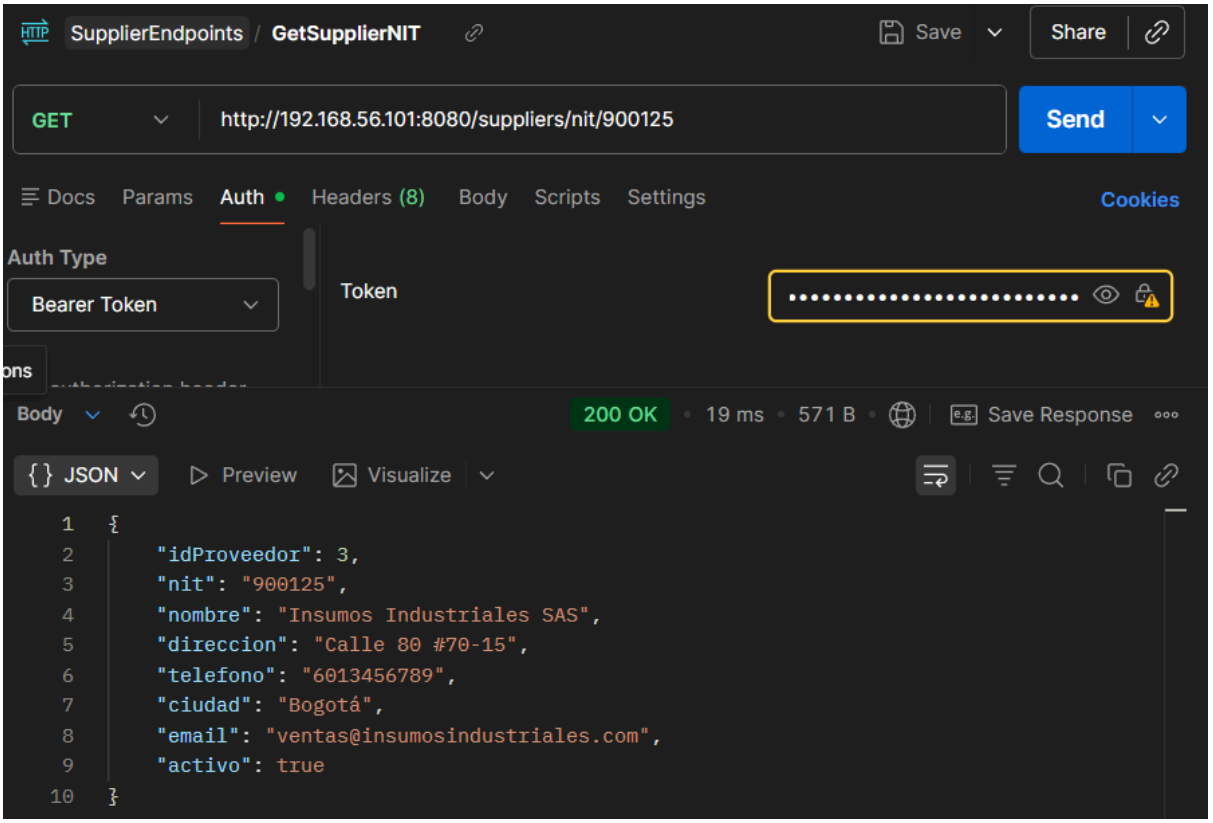

**Listar todos los proveedores:**

GET a http://192.168.56.101:8080/suppliers.



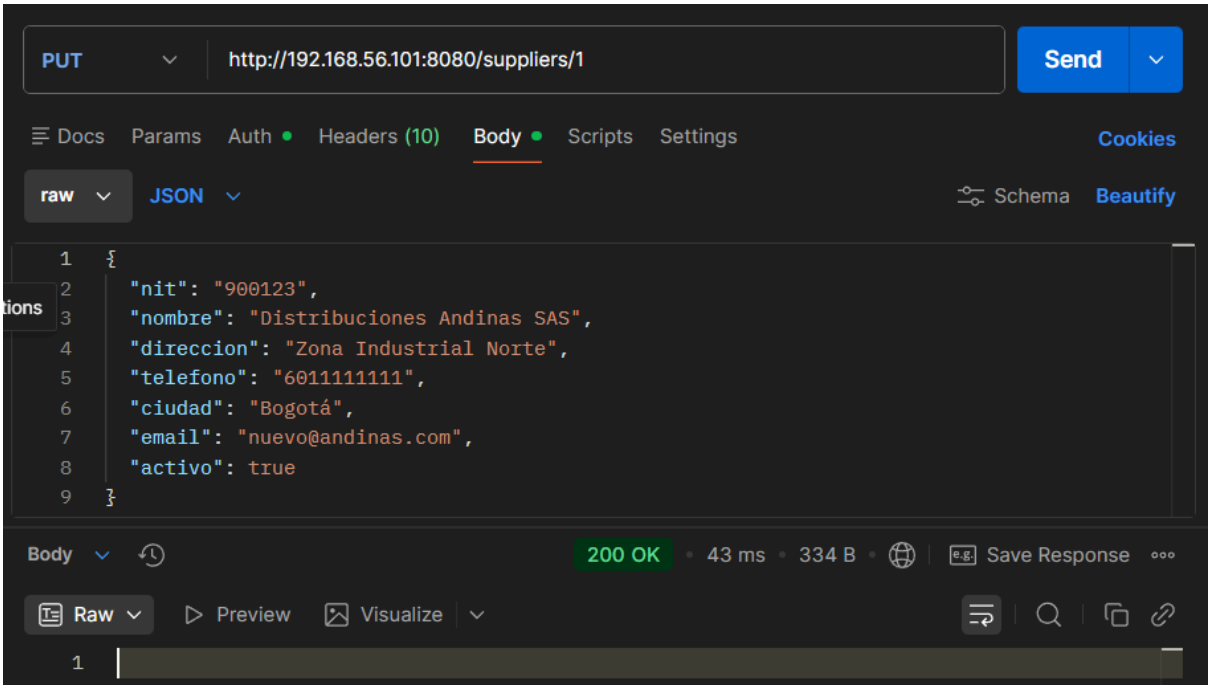

**Buscar proveedor por NIT:**

GET a http://192.168.56.101:8080/suppliers/nit/{nit}.



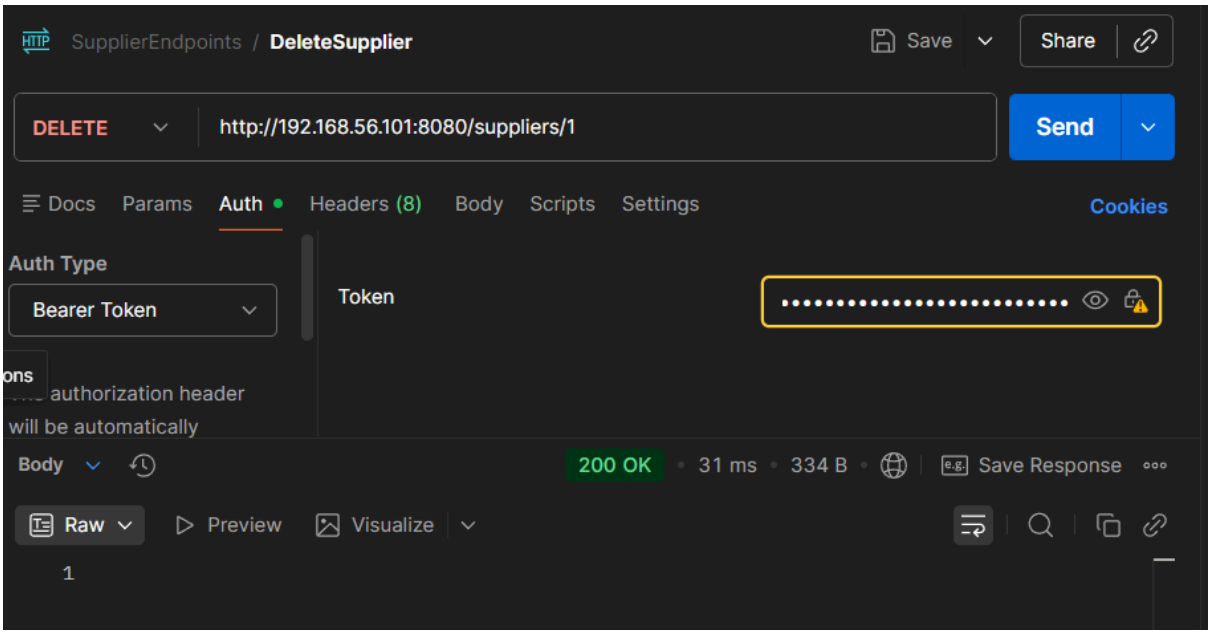

**Actualizar un proveedor:**

PUT a http://192.168.56.101:8080/suppliers/{id}.



**Eliminar un proveedor:**

DELETE a http://192.168.56.101:8080/suppliers/{id}.




## 1.4.4 Catalog Service – Gestión de productos

**Crear un producto manualmente:**

POST a http://192.168.56.101:8080/catalog con los datos del producto.

POST http://192.168.56.101:8080/catalog Send

Docs Params Auth Headers (10) Body Scripts Settings Cookies

raw JSON Schema Beautify

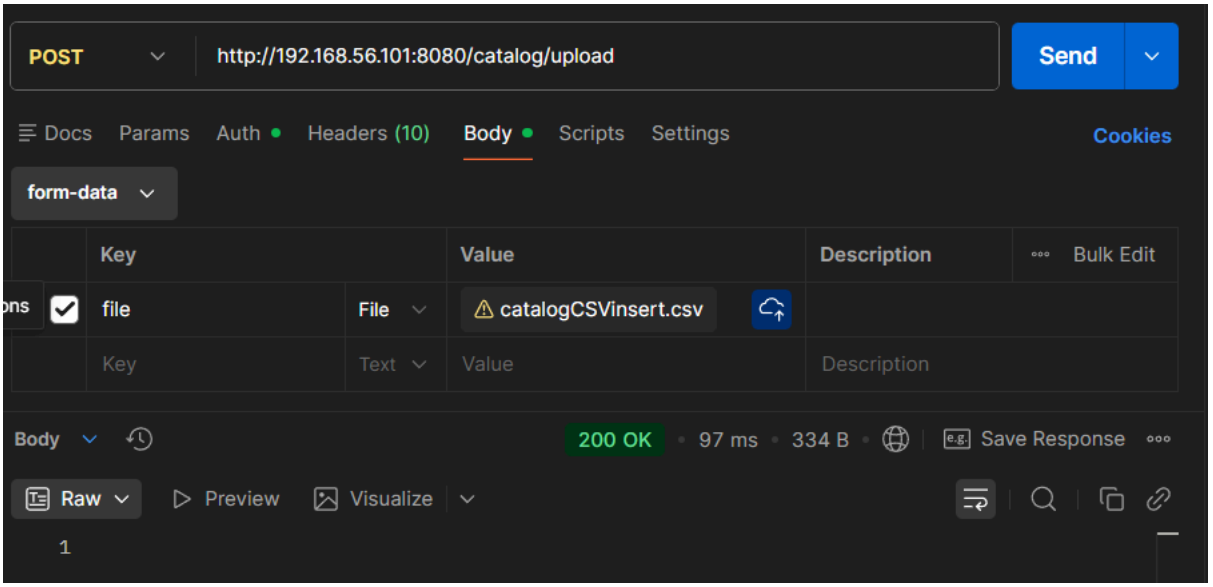
```
{
  "codigo": "P010",
  "nombre": "Café",
  "descripcion": "Café molido premium",
  "precioVenta": 18000,
  "iva": 19,
  "stockActual": 30,
  "stockMinimo": 5,
  "stockMaximo": 80,
  "proveedores": [
    {
      "nit": "900123",
      "precioCompra": 12000
    },
    {
      "nit": "900124",
      "precioCompra": 11500
    }
  ]
}
```

Body 200 OK 573 ms 334 B Save Response

Raw Preview Visualize

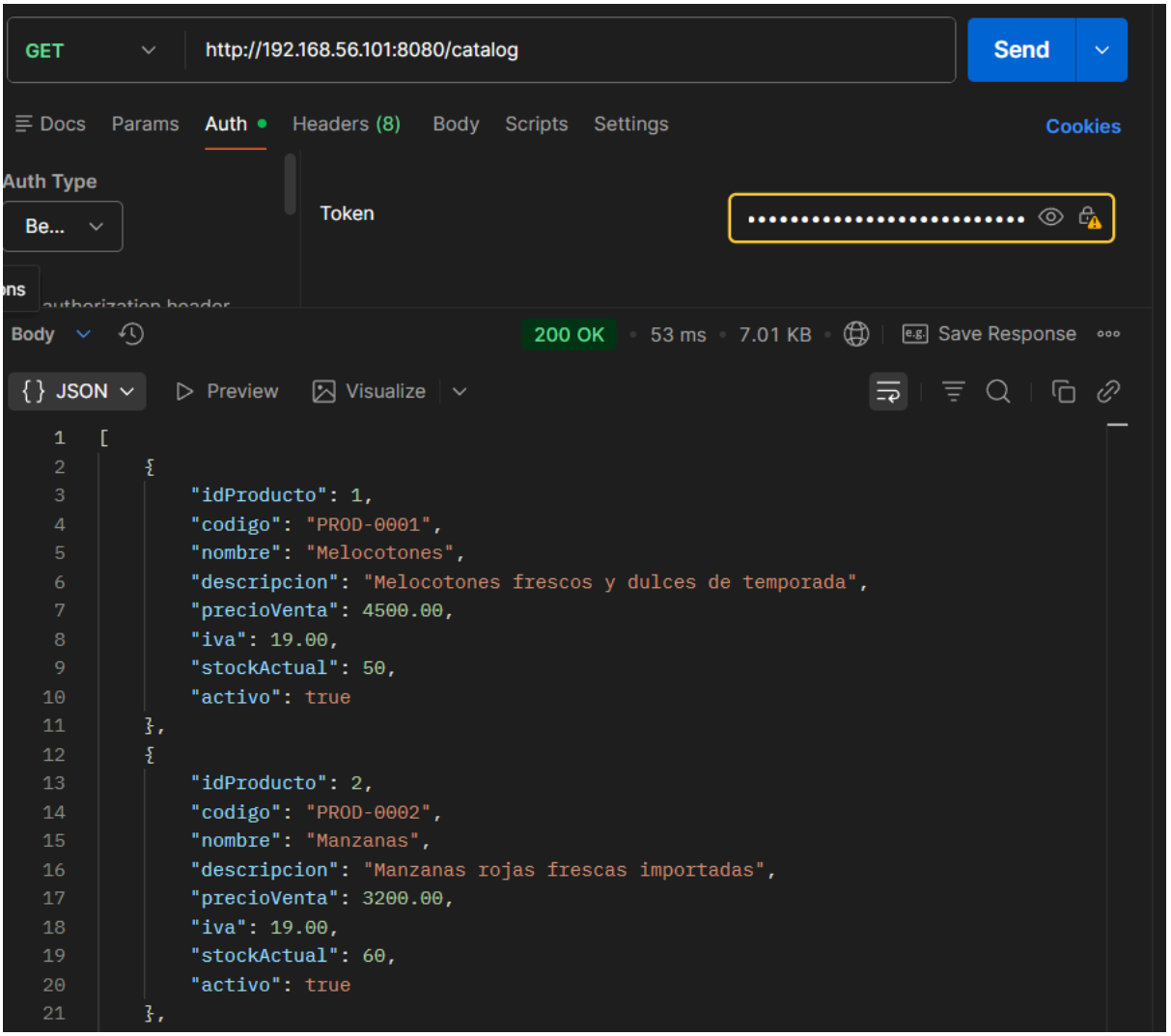
**Cargar productos desde CSV:**

POST a http://192.168.56.101:8080/catalog/upload como multipart/form-data con el archivo CSV en el campo file.




**Listar todos los productos:**

GET a http://192.168.56.101:8080/catalog.




**Actualizar un producto:**

PUT a http:/192.168.56.101:8080/catalog/{id}.

CatalogEndpoints / **UpdateProduct**

Save | Share

PUT http://192.168.56.101:8080/catalog/1 **Send**

Docs | Params | Auth | Headers (10) | Body | Scripts | Settings | Cookies

raw | JSON | Schema | Beautify

```
1  {
2    "codigo": "P001",
3    "nombre": "Melocotones Premium",
4    "descripcion": "Producto actualizado",
5    "precioVenta": 32000,
6    "iva": 19,
7    "stockActual": 70,
8    "stockMinimo": 10,
9    "stockMaximo": 120,
10   "proveedores": [
11     {
12       "nit": "900125",
13       "precioCompra": 15000
14     },
15     {
16       "nit": "900126",
17       "precioCompra": 12500
18     }
19   ]
20 }
```

Body | 200 OK • 18 ms • 334 B | Save Response

Raw | Preview | Visualize

### 1.4.5 Buy Service – Registro de compras a proveedores

**Registrar una compra:**

POST a http://192.168.56.101:8080/buys con el body indicando el ID del proveedor y el detalle de productos con sus cantidades y precios. El servicio valida automáticamente la existencia del proveedor y de cada producto, e incrementa el stock en el catalog-service.

POST http://192.168.56.101:8080/buys Send

Docs Params Auth Headers (10) Body Scripts Settings Cookies

raw JSON Schema Beautify

```
{
  "idProveedor": 3,
  "detalles": [
    {
      "codigoProducto": "PROD-0001",
      "cantidad": 20,
      "precioUnitario": 2000
    },
    {
      "codigoProducto": "PROD-0002",
      "cantidad": 20,
      "precioUnitario": 2000
    }
  ]
}
```

Body 200 OK 462 ms 679 B Save Response

{} JSON Preview Visualize

```
{
    "idCompra": 11,
    "numeroCompra": "COMP-00011",
    "idProveedor": 3,
    "fecha": "2026-03-14T18:34:36.8973482",
    "total": 80000,
    "estado": "CREADA",
    "detalles": [
        {
            "codigoProducto": "PROD-0001",
            "cantidad": 20,
            "precioUnitario": 2000,
            "total": 40000
        },
        {
            "codigoProducto": "PROD-0002",
            "cantidad": 20,
            "precioUnitario": 2000,
            "total": 40000
        }
    ]
}
```

**Listar todas las compras:**

GET a http://192.168.56.101:8080/buys.

GET http://192.168.56.101:8080/buys Send

Docs Params Auth Headers (8) Body Scripts Settings Cookies

Auth Type

Be...

Token

Body 200 OK 44 ms 4.57 KB Save Response

JSON Preview Visualize


```
[
    {
        "idCompra": 1,
        "numeroCompra": "COMP-00001",
        "idProveedor": 1,
        "fecha": "2026-03-14T22:57:27",
        "total": 83000.00,
        "estado": "RECIBIDA",
        "detalles": [
            {
                "codigoProducto": "PROD-0001",
                "cantidad": 10,
                "precioUnitario": 3000.00,
                "total": 30000.00
            },
            {
                "codigoProducto": "PROD-0002",
                "cantidad": 15,
                "precioUnitario": 2200.00,
                "total": 33000.00
            },
            {
                "codigoProducto": "PROD-0003",
                "cantidad": 20,
                "precioUnitario": 1000.00,
                "total": 20000.00
            }
        ]
    },
```


### 1.4.6 Sale Service – Registro de ventas

**Registrar una venta:**

POST a http://192.168.56.101:8080/sales con el body indicando el ID del cliente, el ID del usuario que realiza la venta, los productos con sus códigos y cantidades, y el método de pago. El servicio calcula automáticamente el IVA (19%) y el total final.

POST http://192.168.56.101:8080/sales Send

Docs Params Auth Headers (10) Body Scripts Settings Cookies

raw JSON Schema Beautify

```
{
  "idCliente": 2,
  "idUsuario": 1,
  "detalles": [
    {
      "codigoProducto": "PROD-0001",
      "cantidad": 15
    },
    {
      "codigoProducto": "PROD-0003",
      "cantidad": 15
    }
  ],
  "pagos": [
    {
      "idMetodo": 2
    }
  ]
}
```

Body 200 OK 431 ms 768 B Save Response

JSON Preview Visualize

```
{
    "idVenta": 2,
    "numeroVenta": "COMP-000002",
    "idCliente": 2,
    "idUsuario": 1,
    "fecha": null,
    "totalBruto": 498000.00,
    "totalIva": 94620.0000,
    "totalFinal": 592620.0000,
    "estado": null,
    "detalles": [
        {
            "codigoProducto": "PROD-0001",
            "cantidad": 15,
            "precioUnitario": 32000.00,
            "total": 480000.00
        },
        {
            "codigoProducto": "PROD-0003",
            "cantidad": 15,
            "precioUnitario": 1200.00,
```

**Listar todas las ventas:**

GET a http://192.168.56.101:8080/sales.

GET http://192.168.56.101:8080/sales Send

Docs Params Auth Headers (8) Body Scripts Settings Cookies

Auth Type: Be...

Token: ••••••••••••••••••••••••••••••

Body 200 OK 41 ms 1.09 KB Save Response

JSON Preview Visualize


```
[
    {
        "idVenta": 1,
        "numeroVenta": "VEN-001",
        "idCliente": 1,
        "idUsuario": 1,
        "fecha": "2026-03-14T22:57:28",
        "totalBruto": 30000.00,
        "totalIva": 5700.00,
        "totalFinal": 35700.00,
        "estado": "PAGADA",
        "detalles": [
            {
                "codigoProducto": "PROD-0001",
                "cantidad": 1,
                "precioUnitario": 30351.00,
                "total": 30351.00
            }
        ],
        "pagos": [
            {
                "idMetodo": 1,
                "monto": 35700.00
            }
        ]
    },
```


**Reporte de ventas por cliente:**

GET a http://192.168.56.101:8080/sales/report/client/{idCliente}.

GET http://192.168.56.101:8080/sales/report/client/2 Send

Docs Params Auth Headers (10) Body Scripts Settings Cookies

Auth Type

Be...

Token

authorization header will be automatically generated when you send

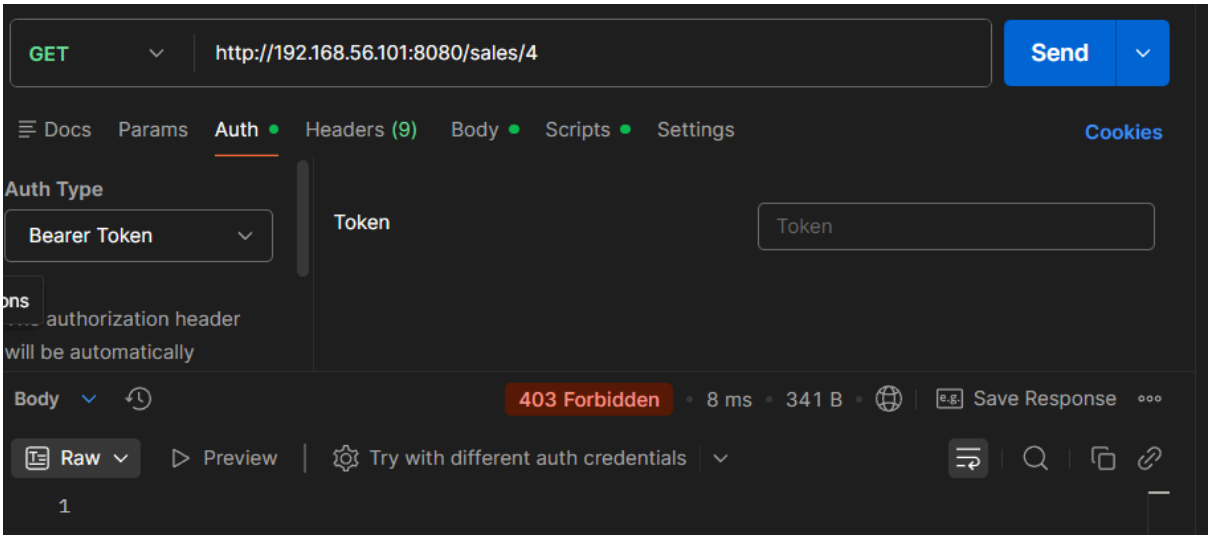
Body 200 OK 28 ms 796 B Save Response

JSON Preview Visualize


```
[
    {
        "idVenta": 2,
        "numeroVenta": "COMP-000002",
        "idCliente": 2,
        "idUsuario": 1,
        "fecha": "2026-03-14T18:35:37",
        "totalBruto": 498000.00,
        "totalIva": 94620.00,
        "totalFinal": 592620.00,
        "estado": "COMPLETADA",
        "detalles": [
            {
                "codigoProducto": "PROD-0001",
                "cantidad": 15,
                "precioUnitario": 32000.00,
                "total": 480000.00
            },
            {
                "codigoProducto": "PROD-0003",
                "cantidad": 15,
                "precioUnitario": 1200.00,
                "total": 18000.00
            }
        ],
        "pagos": [
            {
                "idMetodo": 2,
                "monto": 592620.00
            }
        ]
    }
```


### 1.4.7 Prueba de seguridad – Acceso sin token

Para verificar que la seguridad funciona correctamente, se puede intentar acceder a cualquier endpoint protegido sin el header de autorización. El Gateway debe responder con HTTP 403 Forbidden.


GET
http://192.168.56.101:8080/sales/4
Send
Docs
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
Auth Type
Bearer Token
Token
Token
authorization header
will be automatically
Body
403 Forbidden
8 ms
341 B
Save Response
Raw
Preview
Try with different auth credentials
1

# 2. Funcionalidad de Microservicios e Inconvenientes Superados

## 2.1 ¿Qué es un Microservicio?

Un microservicio es una unidad de software autónoma y altamente especializada que hace una sola cosa bien. En lugar de construir una aplicación que lo maneja todo, la idea es dividir ese sistema en partes independientes, donde cada parte tiene su propia lógica, su propia base de datos y se comunica con las demás únicamente a través de interfaces bien definidas, generalmente APIs REST.

En el caso del proyecto, en lugar de tener un único programa que registra usuarios, gestiona productos, procesa ventas y controla proveedores todo al tiempo, se construyeron ocho servicios separados, cada uno en su propia máquina virtual. El servicio de clientes solo sabe gestionar clientes. El servicio de ventas solo procesa ventas. Si la VM del módulo de proveedores falla, el resto del sistema sigue operando con normalidad.

Lo que más incentivo a desarrollar este sistema con una arquitectura de microservicios es que si mañana la tienda incrementa sus ventas y el módulo de ventas toca escalarlo, se puede escalar únicamente esa VM del módulo respectivo sin tocar el resto. Si hay que corregir un bug en la gestión de proveedores, se genera solo ese WAR sin arriesgar el sistema completo. Además, distintos miembros del equipo pueden trabajar en paralelo sobre servicios distintos sin bloquearse entre sí, lo cual mejora la eficiencia de trabajo.

## 2.2 Microservicio Eureka Server

### VM 1

- **Puerto:** 8761
- **Base de datos:** N/A
- **Tecnología clave:** Spring Cloud Netflix Eureka

### Funcionalidad

El Eureka Server actúa como el registro principal de todos los microservicios. Cuando cualquier servicio arranca en su VM correspondiente, lo primero que hace es registrarse en Eureka con su nombre/ID, la IP de su VM y su puerto. A partir de ese momento, cuando otro servicio necesite comunicarse con él, no necesita conocer la IP directa, sino simplemente consultar en Eureka y obtener la dirección.

Esto se llama Service Discovery y es fundamental en una infraestructura con múltiples VMs porque las IPs pueden cambiar y el sistema sigue funcionando sin reconfiguración manual. En este proyecto, el API Gateway también consulta Eureka usando balanceo de carga (lb://) para enrutar las peticiones a los servicios correctos en sus respectivas VMs.

### Inconvenientes superados

**Orden de arranque estricto y comunicación entre VMs:** Si cualquier microservicio arranca antes que Eureka, intenta registrarse en la IP de la VM 1 en un puerto que aún no responde, generando excepciones de conexión. Además, fue necesario asegurarse de que todas las VMs estuvieran en la misma red local y pudieran comunicarse entre sí.

**Solución:** Se estableció como protocolo arrancar siempre la VM 1 primero y esperar a que Eureka esté completamente operativo antes de arrancar las demás VMs. Spring Boot reintenta el registro automáticamente cada 30 segundos.

## 2.3 Microservicio API Gateway

**VM 2**

- **Puerto:** 8080 (único puerto expuesto al frontend y Postman)
- **Base de datos:** N/A
- **Tecnología clave:** Spring Cloud Gateway, JWT

### Funcionalidad

El API Gateway es la puerta de entrada única del sistema. Todo el tráfico externo (tanto desde el frontend React como desde Postman) entra exclusivamente por la IP de la VM 2 en el puerto 8080. El Gateway decide a qué VM y microservicio enrutar cada petición según el path, usando Eureka para resolver las IPs dinámicamente.

| Path | Destino |
|---|---|
| /auth/** | VM 3 authentication-service |
| /admin/** | VM 3 authentication-service |
| /clients/** | VM 4 client-service |
| /suppliers/** | VM 5 supplier-service |

| | |
|---|---|
| /catalog/** | VM 6 catalog-service |
| /buys/** | VM 7 buy-service |
| /sales/** | VM 8 sale-service |
| /foreignclient/** | VM 4 client-service (ruta interna entre servicios) |
| /foreigncatalog/** | VM 6 catalog-service (ruta interna entre servicios) |
| /foreignsuppliers/** | VM 5 supplier-service (ruta interna entre servicios) |

Además de enrutar, el Gateway valida el token JWT en cada petición externa, por eso mismo responde 403 si el token expiró hace más de una hora o no es válido. También agrega automáticamente el header X-Gateway-Secret a todas las peticiones internas, permitiendo que los microservicios distingan el tráfico legítimo del Gateway del tráfico directo no autorizado.

### Inconvenientes superados

**Comunicación reactiva vs. bloqueante:** Spring Cloud Gateway está construido sobre Spring WebFlux, pero los microservicios de negocio usan Spring MVC. Esto generó errores al intentar usar dependencias bloqueantes dentro del Gateway.

**Solución:** Se mantuvo el Gateway completamente reactivo, sin acceso a base de datos, delegando toda lógica de negocio a los microservicios en sus respectivas VMs. El Gateway solo válida JWT y enruta.

**CORS con el frontend React (Vite):** El navegador bloqueaba las peticiones del frontend al Gateway por política de Same-Origin al correr en distintos orígenes.

**Solución:** Se implementó una clase CorsConfig centralizada en el Gateway que permite el origen del frontend. Al estar centralizada en el único punto de entrada, no fue necesario configurar CORS en cada microservicio en sus respectivas VMs.

## 2.4 Microservicio: Authentication Service

**VM 3**

- **Puerto:** 8081
- **Base de datos:** tienda_aut_db (VM 3, puerto 3306)

- **Tablas:** usuarios, roles, usuario_rol
- **Tecnología clave:** Spring Security, JWT, BCrypt

## Funcionalidad

Este servicio maneja la autenticación de usuario y la autorización. Está organizado en dos controladores distintos.

**AuthController (Endpoints públicos):**

| Método | Endpoint | Descripción |
|---|---|---|
| POST | /auth/login | Valida credenciales contra la base de datos local de la VM 3 y retorna un JWT firmado con roles embebidos |
| POST | /auth/signup | Registra un nuevo usuario |

**AdminController (Endpoints protegidos que requieren ADMIN):**

| Método | Endpoint | Descripción |
|---|---|---|
| GET | /admin/users | Lista todos los usuarios |
| GET | /admin/users/{id} | Consulta un usuario por ID |
| POST | /admin/users | Crea un nuevo usuario con roles asignados |
| PUT | /admin/users/{id} | Actualiza datos de un usuario |
| DELETE | /admin/users/{id} | Elimina permanentemente un usuario |
| DELETE | /admin/users/{id}/act | Desactiva un usuario (soft delete) |
| PATCH | /admin/users/{id}/roles | Actualiza los roles sin tocar datos personales |
| PATCH | /admin/users/{id}/credentials | Actualiza username y/o contraseña |

El usuario aparte de almacenar solo usuario y contraseña también guarda datos como el nombre, apellido, activo (soft delete), fecha_creacion y un sistema de roles con tabla pivote que permite asignar múltiples roles por usuario (ADMIN, GERENTE, INVENTARIO, CAJERO). Las contraseñas se almacenan siempre hasheadas con BCrypt. El JWT generado incluye el ID del usuario, su username y sus roles, con expiración de 1 hora.

El usuario inicial del documento (admininicial / admin123456) se carga ejecutando el script insert.sql directamente sobre MySQL en la VM 3 antes del primer arranque del servicio.

### Inconvenientes superados

**Filtro de seguridad bloqueando llamadas internas entre VMs:** Cada microservicio tiene su propio filtro JWT. Cuando el buy-service en la VM 7 llama al catalog-service en la VM 6 para actualizar stock, esa petición pasa por el filtro JWT del catalog-service, pero el buy-service no tiene el JWT del usuario original.

**Solución:** El Gateway en VM 2 añade el header X-Gateway-Secret a toda petición que reenvía entre VMs. Cada microservicio verifica primero ese header, y si está presente y es correcto, deja pasar la petición sin exigir JWT. Solo las peticiones externas requieren token Bearer.

**Tabla con clave compuesta (@EmbeddedId) no se integra en JPA:** La relación muchos-a-muchos entre usuarios y roles requiere la tabla usuario_rol con PK compuesta. Mapear esto con JPA usando @EmbeddedId y @MapsId generó errores de cascada en las primeras iteraciones.

**Solución:** Se creó la clase UserRoleId con @Embeddable. La entidad UserRole usa @EmbeddedId y dos @ManyToOne con @MapsId. La carga EAGER en la lista de roles garantiza que siempre estén disponibles al construir el JWT.

## 2.5 Microservicio: Client Service

### VM 4

- **Puerto:** 8082
- **Base de datos:** tienda_cli_db (VM 4, puerto 3306)
- **Tablas:** clientes
- **Roles con acceso:** ADMIN, GERENTE

### Funcionalidad

Administra el ciclo de vida completo de los clientes. El modelo incluye cédula única, nombre, apellido, dirección, teléfono, email y campo activo para borrado lógico. Tiene dos

controladores: uno externo que exige JWT con rol válido, y uno interno (/foreignclient/**) que acepta peticiones de otros microservicios en otras VMs mediante X-Gateway-Secret.

| Método | Endpoint | Descripción |
|---|---|---|
| GET | /clients | Lista todos los clientes |
| GET | /clients/{id} | Obtiene un cliente por ID |
| POST | /clients | Registra un nuevo cliente |
| PUT | /clients/{id} | Actualiza datos de un cliente |
| DELETE | /clients/{id} | Elimina permanentemente un cliente |
| DELETE | /clients/{id}/act | Desactiva un cliente (soft delete) |
| GET | /clients/active-clients | Lista clientes activos con ID y cédula |
| GET | /foreignclient/{id} | Consulta interna desde VM 8 (sale-service) para validar el cliente antes de registrar una venta |

### Inconvenientes superados

**Cédula duplicada retornaba error 500 genérico:** Al intentar crear un cliente con una cédula ya registrada, la violación del constraint UNIQUE de MySQL llegaba al cliente como un error 500 sin mensaje útil.

**Solución:** Se implementó ResourceAlreadyExistsException que el servicio lanza antes de intentar guardar, tras verificar si ya existe esa cédula en la base de datos local de la VM 4. Un @RestControllerAdvice global la convierte en HTTP 409 Conflict con mensaje legible.

## 2.6 Microservicio: Supplier Service

### VM 5

- **Puerto:** 8083
- **Base de datos:** tienda_sup_db (VM 5, puerto 3306)
- **Tablas:** proveedores

- **Roles con acceso:** ADMIN, GERENTE, INVENTARIO

## Funcionalidad

Gestiona el directorio de proveedores. El modelo incluye NIT único, nombre, dirección, teléfono, ciudad, email y campo activo. Tiene un controlador externo con JWT y un controlador interno (/foreignsuppliers/**) para comunicación desde otras VMs.

| Método | Endpoint | Descripción |
|---|---|---|
| GET | /suppliers | Lista todos los proveedores |
| GET | /suppliers/{id} | Busca por ID interno |
| GET | /suppliers/nit/{nit} | Busca por NIT del proveedor |
| POST | /suppliers | Registra un nuevo proveedor |
| PUT | /suppliers/{id} | Actualiza datos de un proveedor |
| DELETE | /suppliers/{id} | Elimina permanentemente un proveedor |
| DELETE | /suppliers/{id}/act | Desactiva un proveedor (soft delete) |
| GET | /suppliers/active-nits | Lista NIT y nombre de proveedores activos |
| GET | /foreignsuppliers/{id} | Consulta interna desde VM 7 (buy-service) para validar el proveedor antes de registrar una compra |

## Inconvenientes superados

**NIT como identificador de negocio vs. ID técnico:** Lo ideal para un software de este tipo es buscar proveedores por NIT directamente, pero JPA trabaja con el ID autoincremental internamente.

**Solución:** Se implementaron /suppliers/{id} para operaciones internas entre VMs y /suppliers/nit/{nit} para búsquedas desde el frontend. Las relaciones entre servicios usan siempre el ID, pero desde el frontend se maneja el NIT del proveedor.

## 2.7 Microservicio: Catalog Service

**VM 6**

- **Puerto:** 8084
- **Base de datos:** tienda_cat_db (VM 6, puerto 3306)
- **Tablas:** productos
- **Roles con acceso:** ADMIN, GERENTE, INVENTARIO

### Funcionalidad

Administra el catálogo de productos y es el único responsable del stock de todo el sistema. Tanto el buy-service en la VM 7 como el sale-service en la VM 8 modifican el stock enviando peticiones a este servicio a través del Gateway, siendo la VM 6 la que aloja esta información. Esto evita inconsistencias que ocurrirían si cada VM mantuviera su propia copia del stock.

Aquí cada producto se maneja con código autogenerado (PROD-XXXX), descripción, IVA como BigDecimal para precisión decimal, stock_actual, stock_minimo y stock_maximo.

| Método | Endpoint | Descripción |
|---|---|---|
| GET | /catalog | Lista todos los productos |
| GET | /catalog/{id} | Obtiene un producto por ID |
| POST | /catalog | Crea un producto manualmente |
| PUT | /catalog/{id} | Actualiza datos de un producto |
| DELETE | /catalog/{id} | Elimina permanentemente un producto |
| DELETE | /catalog/{id}/act | Desactiva un producto (soft delete) |
| POST | /catalog/upload | Carga masiva desde CSV (multipart/form-data) |
| GET | /foreigncatalog/codigo/{codigo} | Consulta interna por código; usada desde VM 7 y VM 8 |

| | | |
|---|---|---|
| PUT | /foreigncatalog/codigo/{codigo}/stock | Actualiza stock. Esta es invocada desde VM 7 (compras) y VM 8 (ventas) |

### Inconvenientes superados

**Actualización de stock con cantidad negativa sumaba en lugar de restar:** Cuando el sale-service en la VM 8 enviaba una cantidad negativa para descontar stock vendido, el servicio la sumaba igualmente porque el método siempre aplicaba adición.

**Solución:** Se rediseñó updateStock para aceptar cantidades de cierta forma, positiva incrementa (compra desde VM 7), negativa decrementa (venta desde VM 8). Se agregó validación para lanzar excepción si el stock resultante quedaría negativo.

**Caracteres especiales (tildes, ñ) en nombres del CSV:** Nombres como "Champiñones" retornaban caracteres basura porque el lector no especificaba el encoding.

**Solución:** Se pasó StandardCharsets.UTF_8 explícitamente al construir el InputStreamReader con el new InputStreamReader(file.getInputStream(), StandardCharsets.UTF_8).

## 2.8 Microservicio: Buy Service

**VM 7**

- **Puerto:** 8085
- **Base de datos:** tienda_buy_db (VM 7, puerto 3306)
- **Tablas:** compras, detalle_compras
- **Roles con acceso:** ADMIN, GERENTE, INVENTARIO

### Funcionalidad

La inclusión de este servicio responde a una necesidad real, la cual es que antes de vender productos, la tienda debe comprarlos a sus proveedores y ese stock debe ingresar al sistema con trazabilidad completa. El flujo al crear una compra es validar proveedor consultando a la VM 5 (supplier-service), y posteriormente validar cada producto consultando a la VM 6 (catalog-service), para luego incrementar stock de cada ítem en la VM 6, y después persistir la compra con número autogenerado (COMP-XXXXX) en la base de datos local de la VM 7.

| Método | Endpoint | Descripción |
|---|---|---|

| | | |
|---|---|---|
| GET | /buys | Lista todas las compras con sus detalles |
| GET | /buys/{id} | Obtiene una compra específica |
| POST | /buys | Registra una compra, válida proveedor en VM 5 y productos en VM 6, actualiza stock en VM 6 automáticamente |
| DELETE | /buys/{id} | Elimina una compra |

### Inconvenientes superados

**Feign traducía el 404 de otras VMs como excepción genérica:** Cuando la VM 6 o la VM 5 retornaban HTTP 404 porque un recurso no existía, Feign lo envolvía en una excepción genérica difícil de diferenciar de otros errores de comunicación entre VMs.

**Solución:** Se capturó la subclase específica feign.FeignException.NotFound, permitiendo distinguir "recurso no encontrado" de cualquier otro fallo de red entre VMs y retornar un mensaje de error descriptivo al cliente.

**Inconsistencia de stock entre VMs si el procesamiento falla a mitad:** Si la VM 7 actualizaba el stock en la VM 6 para los primeros productos y luego encontraba uno inválido, los primeros ya tenían stock incrementado pero la compra no quedó registrada en la VM 7.

**Solución:** Primero se validan todos los productos y el proveedor sin modificar nada en ninguna VM, y solo cuando todas las validaciones pasan se actualiza el stock en la VM 6 y se persiste la compra en la VM 7.

## 2.9 Microservicio: Sale Service

### VM 8

- **Puerto:** 8086
- **Base de datos:** tienda_sal_db (VM 8, puerto 3306)
- **Tablas:** ventas, detalle_ventas, metodos_pago, pagos
- **Roles con acceso:** ADMIN, GERENTE, INVENTARIO, CAJERO

### Funcionalidad

Implementa el flujo completo de venta al cliente, que es el proceso central del negocio. Este permite múltiples productos por venta sin límite, calcula IVA automáticamente y soporta múltiples métodos de pago por venta a través de las entidades Payment y PaymentMethod.

Aquí se intenta validar cliente consultando a la VM 4 (client-service), luego consultar precio y stock de cada producto en la VM 6 (catalog-service), para posteriormente calcular totales con 19% IVA, después persistir venta y detalles en la base de datos local de la VM 8, para finalmente descontar stock de cada producto enviando petición a la VM 6.

| Método | Endpoint | Descripción |
|---|---|---|
| POST | /sales | Crea una venta, válida el cliente en VM 4 y productos en VM 6. También descuenta stock en VM 6 generando un código COMP-XXXXXX |
| GET | /sales | Lista todas las ventas con detalles y pagos |
| GET | /sales/{id} | Obtiene una venta específica |
| DELETE | /sales/{id} | Elimina una venta |
| GET | /sales/report/client/{idCliente} | Ventas filtradas por cliente, y valida existencia del cliente en VM 4 |

## Inconvenientes superados

**Número de venta consecutivo con riesgo de duplicado en concurrencia:** El número se genera consultando el último registro y sumando 1. Con múltiples usuarios simultáneos, dos peticiones podrían leer el mismo "último" número antes de que cualquiera guarde.

**Solución:** El método createSale está anotado con @Transactional y la columna numero_venta tiene restricción UNIQUE en MySQL, lo que rechaza duplicados con una excepción manejable como red de seguridad.

**Dos FeignClients al mismo destino generaban conflicto de beans:** CatalogClient y CustomerClient usaban name = "api-gateway" sin diferenciarse, y Spring intentaba registrar dos beans con el mismo nombre al inicializar.

**Solución:** Se añadió contextId = "catalogClient" y contextId = "customerClient" en las respectivas anotaciones @FeignClient, indicándole a Spring que son beans distintos aunque apunten al mismo Gateway en la VM 2.

# 3. Arquitectura Utilizada e Implementación

## 3.1 ¿Qué arquitectura se utilizó?

El proyecto implementa una arquitectura de microservicios con el patrón API Gateway y Service Discovery, complementada con el patrón de seguridad JWT (JSON Web Token) para autenticación. Cada microservicio está desplegado en una máquina virtual Ubuntu 22.04.5 LTS independiente, con su propio servidor Tomcat y su propia instancia de MySQL.

Originalmente este proyecto utilizaba una arquitectura monolítica con JavaEE, JSF y JSP con un único servidor Tomcat que contiene toda la lógica del sistema, con una única base de datos compartida. Aunque es válida y funcional, presenta limitaciones importantes, ya que si falla el servidor, se cae todo el sistema. También está el caso de si hay que actualizar un módulo, hay que desplegar la aplicación entera.

La arquitectura de microservicios con VMs independientes resuelve estos problemas dividiendo el sistema en componentes que no solo son independientes a nivel de código, sino también a nivel de infraestructura física, debido a que cada servicio tiene su propio sistema operativo, su propia memoria, su propio procesador asignado y su propia base de datos. Si la VM 8 (ventas) falla, las VMs de clientes, productos y proveedores siguen operando con normalidad.

## 3.2 Componentes de la arquitectura y cómo se implementaron

- **Capa 1 - Registro de servicios (VM 1, Eureka Server):**

El eureka-server corre en la VM 1 en el puerto 8761 y mantiene un registro dinámico de todas las instancias de microservicios disponibles en la red. Cada servicio se registra al arrancar con su nombre lógico, la IP de su VM y su puerto. Cuando el Gateway en la VM 2 necesita enrutar una petición al catalog-service, no tiene la IP de la VM 6 por lo que le pregunta a Eureka y obtiene la dirección actual. Esto se configura en el application.properties de cada servicio apuntando a la IP de la VM 1:

eureka.client.service-url.defaultZone=http://192.168.56.100:8761/eureka

Y en el api-gateway, las rutas usan el lb:// para que Spring Cloud tenga el nombre lógico a través de Eureka:

spring.cloud.gateway.routes[4].uri=lb://catalog-service

- **Capa 2 - Punto de entrada único (VM 2, API Gateway):**

El api-gateway en la VM 2 es el único componente expuesto en el puerto 8080. Todos los demás servicios en sus VMs respectivas solo necesitan ser accesibles dentro de la red interna. El Gateway cumple tres funciones: enrutar peticiones a la VM correcta según el path, validar tokens JWT en peticiones externas, e inyectar el X-Gateway-Secret en peticiones internas entre VMs. Está implementado con Spring Cloud Gateway (WebFlux) y la validación JWT se hace con la librería JJWT de forma completamente stateless, sin consultar ninguna base de datos.

- **Capa 3 - Microservicios de negocio (VMs 3 a 8):**

Cada microservicio sigue la misma estructura. Todos tienen un Controller, Service, Repository, y Entity. Esta es la arquitectura en capas clásica aplicada dentro de cada microservicio.

- Controller expone los endpoints REST y delega al Service.
- Service contiene toda la lógica de negocio, validaciones y orquestación de llamadas a otras VMs.
- Repository interactúa con la base de datos local usando Spring Data JPA.
- Entity mapea las tablas de la base de datos local con anotaciones JPA.
- DTO desacopla el modelo de base de datos de lo que se expone en la API, evitando filtrar datos sensibles como contraseñas hasheadas.

- **Capa 4 – Bases de datos independientes (MySQL local en cada VM):**

Cada microservicio de negocio (VMs 3 a 8) tiene su propia instancia de MySQL corriendo localmente en el puerto 3306. Hay una base de datos por servicio, y ningún servicio puede acceder directamente a la base de datos de otra VM, forzando que toda comunicación entre servicios pase por las APIs REST a través del Gateway. Los scripts de inicialización (create.sql e insert.sql) se ejecutan en cada VM antes del primer arranque del servicio correspondiente.

## 3.3 Comunicación entre microservicios en distintas VMs

Cuando un microservicio en una VM necesita datos de otro microservicio en otra VM (por ejemplo, sale-service en la VM 8 necesita verificar que el cliente existe en la VM 4), no accede directamente a la base de datos de esa VM. En cambio, realiza una petición HTTP REST a través del API Gateway en la VM 2. Para esto se usó OpenFeign, una librería de Spring Cloud que permite declarar clientes HTTP como interfaces de Java simples.

```
@FeignClient(name = "api-gateway", contextId = "customerClient")

public interface CustomerClient {
```

```
    @GetMapping("/foreignclient/{id}")
    ClientDTO getClientById(@PathVariable("id") Long id);
}
```

Feign se encarga de construir la petición HTTP hacia el Gateway en la VM 2, que a su vez la enruta a la VM 4. Esto hace que llamar a un servicio en otra VM sea tan simple como llamar a un método local desde el código.

Las rutas /foreign* son rutas dedicadas a la comunicación interna entre VMs. No exigen JWT pero sí el header X-Gateway-Secret, separando claramente el tráfico externo (usuarios del frontend) del tráfico interno de las VMs comunicándose entre sí.

## 3.4 Seguridad de la arquitectura

El modelo de seguridad implementado funciona de la siguiente forma:

1. El usuario hace login en /auth/login a través del Gateway en la VM 2, que enruta la petición al authentication-service en la VM 3.
2. La VM 3 valida las credenciales contra su MySQL local y genera un JWT firmado con HMAC-SHA256. El JWT contiene el ID del usuario, su username y sus roles, con expiración de 1 hora.
3. En cada petición posterior, el cliente envía el JWT en el header Authorization: Bearer <token>.
4. El Gateway en la VM 2 intercepta la petición, válida la firma del JWT usando el secret compartido configurado en todos los servicios, y si es válido la reenvía a la VM correspondiente añadiendo el X-Gateway-Secret.
5. El microservicio solicitado verifica el X-Gateway-Secret para confirmar que la petición pasó por el Gateway legítimo, luego extrae los roles del JWT para aplicar control de acceso con @PreAuthorize.

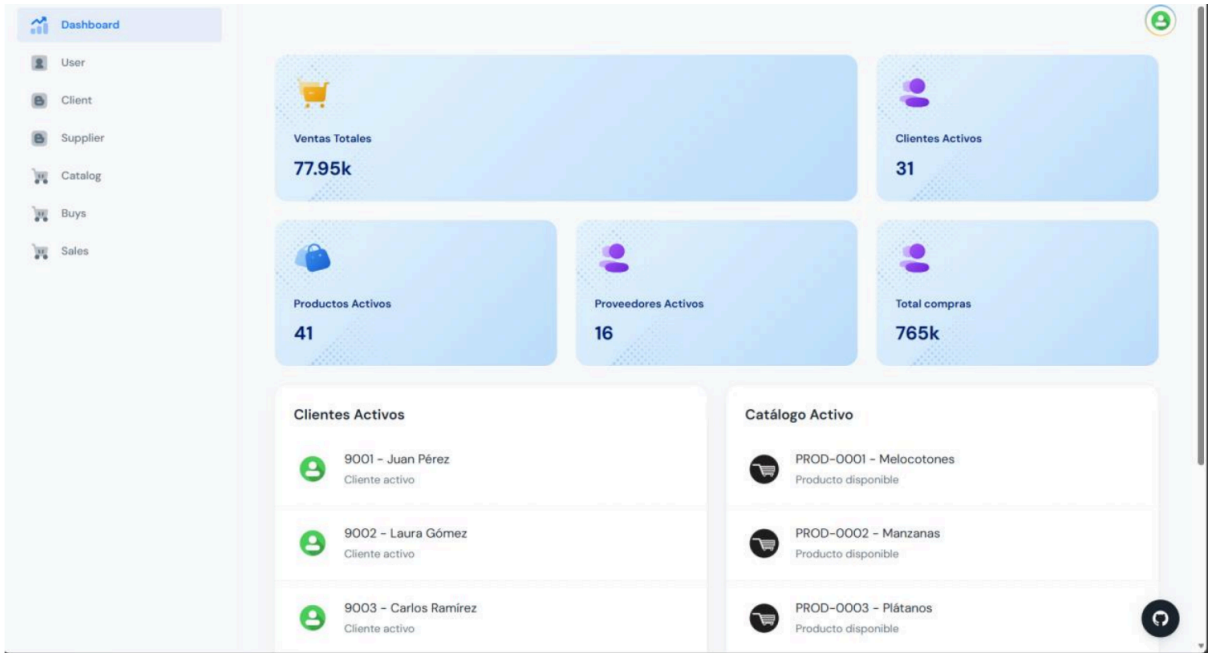
## 3.5 Frontend con React Vite

El frontend de la aplicación es una SPA (Single Page Application) desarrollada con React y empaquetada con Vite. Originalmente se planteaban interfaces JSP/JSF embebidas en el servidor, sin embargo, se determinó que tener React como front era más cómodo y eficaz.

React se eligió porque es la librería de UI más utilizada actualmente con amplio soporte. Vite se eligió como herramienta de empaquetado porque ofrece tiempos de arranque y recarga en desarrollo significativamente más rápidos que las alternativas anteriores como Create React App, gracias a su uso de ES Modules nativos del navegador.

El frontend se comunica exclusivamente con el API Gateway en la VM 2 (http://IP-VM2:8080) mediante peticiones HTTP REST con fetch o axios. Nunca se comunica directamente con ninguna VM de servicio específica, respetando el principio del punto de entrada único. Al hacer login, el frontend almacena el JWT recibido y lo adjunta automáticamente en el header Authorization de todas las peticiones subsiguientes.

Esta separación frontend-backend es el estándar de la industria actual y ofrece ventajas concretas. Por ejemplo, el mismo backend puede ser consumido por una aplicación móvil en el futuro sin ningún cambio, mientras que el frontend puede desplegarse en un servidor de archivos estáticos o CDN independientemente del backend. Esto ayudó a que tanto back como front pueden trabajar y desplegar en paralelo sin interferirse.




## 3.6 Justificación de cambios tecnológicos respecto al documento original

**Java 17 en lugar de JavaSE 11:** Java 17 es más estable al momento del desarrollo. Asimismo, Spring Boot 3 en adelante requiere mínimo Java 17 y no es compatible con versiones anteriores. Además, Java 17 ofrece mejoras de rendimiento y características del lenguaje que simplifican el código.

**Spring Boot 3 en lugar de Spring Boot 2.4.5 con JSF/JSP:** Spring Boot 3 con sus starters simplifica la configuración. JSF y JSP son tecnologías de renderizado en el servidor que generan HTML directamente, lo cual es incompatible con una arquitectura donde el frontend es una aplicación React separada que consume una API REST. Spring Boot con controladores

REST devuelve JSON, lo que permite que cualquier cliente, ya sea React o Postman consuma la API sin cambios en el backend.

**Ubuntu 22.04.5 LTS en lugar de Ubuntu mínimo sin especificación:** Ubuntu Server 22.04.4 LTS es una versión de soporte relativamente reciente de Ubuntu al momento del desarrollo. Provee una instalación limpia con lo necesario para un servidor y con entorno gráfico. Aunque máquinas minimal hubieran beneficiado el rendimiento y recursos para Tomcat y MySQL, esto beneficio la eficiencia en el desarrollo.

**Arquitectura de microservicios con VMs independientes en lugar de monolito en una sola VM:** En vez de un único servidor con toda la aplicación. La arquitectura de microservicios con VMs separadas implica mayor complejidad de infraestructura, pero es la arquitectura que utilizan sistemas reales de comercio como Amazon, Netflix y Mercado Libre. El aislamiento por VM garantiza tolerancia a fallos y transparencia.

**MySQL 8 instalado localmente en cada VM en lugar de una base de datos compartida:** Tener una base de datos por VM implica que ningún servicio puede hacer consultas SQL directas a los datos de otro, toda comunicación es por API. Esto también permite que cada base de datos se configure y optimice de forma independiente según las necesidades de su servicio.

**React con Vite en lugar de JSP/JSF embebido:** JSF y JSP generan HTML en el servidor y lo envían al navegador, acoplando fuertemente el frontend al backend dentro del mismo servidor. En cambio, React genera una aplicación TypeScript que corre enteramente en el navegador del usuario y se comunica con el backend mediante API REST. Esto desacopla completamente las capas de presentación y lógica, permite que el frontend evolucione independientemente del backend, y es el modelo de desarrollo web dominante en la industria actual. Aparte aqui se dispone con materialUI la cual es una librería de diseños bastante amplia.

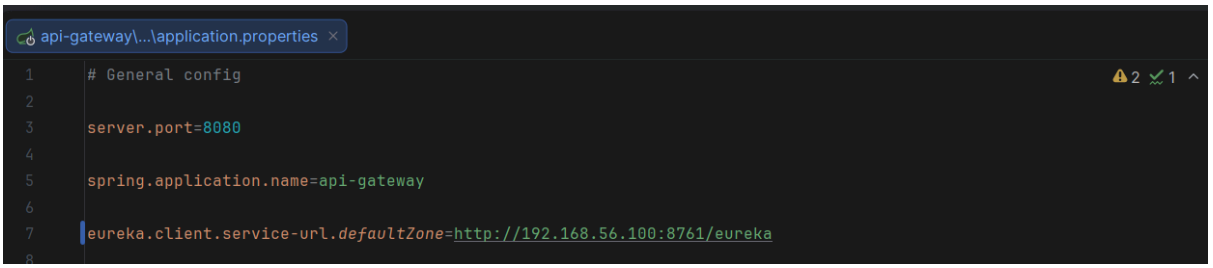
## 3.7 Patrones de diseño aplicados

**API Gateway Pattern:** Un único punto de entrada en la VM 2 que centraliza el enrutamiento entre VMs, la autenticación y el CORS. El cliente solo necesita conocer la IP de la VM 2, por lo que las IPs del resto de VMs están conectadas con el gateway.

**Service Discovery Pattern:** Eureka en la VM 1 permite que los servicios se encuentren dinámicamente por nombre lógico sin IPs hardcodeadas entre VMs. El Gateway usa balanceo de carga a través del prefijo lb://.

**Database per Service Pattern:** Cada VM de negocio (3 a 8) tiene su propia instancia de MySQL. Ningún servicio puede acceder directamente a la base de datos de otra VM, toda comunicación es por API REST a través del Gateway.

**DTO Pattern (Data Transfer Object):** Se usan clases DTO separadas de las entidades JPA para controlar exactamente qué datos se exponen en cada endpoint. La entidad User tiene el campo password hasheado, pero UserResponseDTO no lo incluye, por lo que nunca se expone a través de la API.

**Repository Pattern:** Spring Data JPA implementa el patrón Repository, donde toda la lógica de acceso a la base de datos local de cada VM se encapsula en interfaces, manteniendo el código de negocio desacoplado de la tecnología de persistencia.

**Shared Secret Pattern para comunicación interna entre VMs:** Los servicios se autentican entre sí mediante el header X-Gateway-Secret cuando se comunican a través del Gateway. Esto separa claramente el canal de autenticación externo de JWT de usuarios del canal de autenticación interno del secret compartido entre VMs.